
CYBERSECURITY FOR PROFESSIONALS BOOTCAMP

HANDS-ON LAB

Vulnerability Assessment and Digital Forensics

By: Jason Man

Week 6, Days 1–3 • Browser-Based Lab • Individual Lab

1. Before You Start

This lab runs entirely in your web browser. There is nothing to install, no virtual machine to connect to, no account to create, and no password to request. If you can open a web page and use a spreadsheet, you have everything you need. Read this section fully before Day 1. It takes five minutes and it will save you from every common problem in this lab.

1.1 What you need

Item	Detail
A web browser	Chrome, Edge, Firefox or Safari. Any current version is fine.
A spreadsheet	Excel, Google Sheets or LibreOffice Calc. You will build two tables in it.
A document editor	Word, Google Docs or anything that produces a PDF for your final reports.
This document	All evidence you need is printed inside it. Keep it open beside your browser.

1.2 The three websites you will use

Tool	Address	What you use it for
CyberChef	gchq.github.io/CyberChef	Hashing evidence, decoding a suspicious file, and pulling indicators out of log data. Published by GCHQ. Runs entirely inside your browser - nothing you type is ever uploaded anywhere.
NVD	nvd.nist.gov	The US National Vulnerability Database. You look up which software versions a CVE actually affects. This is how you prove a scanner finding is wrong.
CISA KEV Catalog	cisa.gov/known-exploited-vulnerabilities-catalog	The US government list of vulnerabilities confirmed to be exploited by real attackers. This is the threat intelligence source you re-prioritise against.

If a website will not load

Every piece of evidence you need is printed in this document. The websites confirm what you already have; they do not hold anything you cannot see here.

CyberChef: once the page has loaded once, it keeps working even with your network off. If it will not load at all, Section 9.5 gives you a command built into Windows and macOS that does the same job.

NVD: Appendix A prints the affected-version ranges for every CVE in this lab. Use those and note in your report that you used the printed values.

CISA KEV: Appendix A prints the KEV status and date for every CVE in this lab.

You are never blocked. Move on and keep working.

1.3 How this lab is structured

Session	What you do	What you hand in
Day 1	Triage a real vulnerability scan export, then manually validate the findings that matter.	Severity breakdown, top ten list, five validated findings including at least one false positive.
Day 2	Cross-reference your validated findings against live threat intelligence and re-prioritise, then write the assessment report.	KEV cross-reference table, re-prioritised list, full vulnerability assessment report.
Day 3	One of the hosts you assessed has actually been compromised. Preserve evidence, build a timeline, analyse logs, and find the root cause.	Evidence record, timeline, consolidated IOC list, forensic report.

Days 1 to 2 and Day 3 are one story

The vulnerability you correctly prioritise on Day 2 is the same vulnerability an attacker used on Day 3. If you rush Days 1 and 2, Day 3 will not make sense. If you do them properly, Day 3 will feel obvious.

2. The Scenario

You are a junior security analyst at Meridian Trust Bank. The bank runs vulnerability assessments on a regular cadence across all environments, not just once before launch.

You have been asked to run this cycle's assessment against a representative slice of the bank's infrastructure, validate what the scanner found, and deliver a report the infrastructure team can actually act on.

2.1 The environment you are assessing

Host	Address	Role	Notes
MTB-WEB01	10.50.2.24	Customer portal web server	DMZ, internet facing, high criticality
MTB-APP01	10.50.2.25	Application and file transfer	Legacy FTP still in use
MTB-FILE01	10.50.2.26	Internal file server	Legacy protocols still enabled
MTB-DC01	10.50.2.10	Windows domain controller	Authenticated scan credentials supplied
MTB-DVWA01	10.50.2.30	Test web application	Non-production, deliberately weak

Scope and ethics

Everything in this lab is evidence that has already been collected for you. You are analysing it, not generating it.

Do not point any scanning tool at any host, website or address anywhere. Not the addresses in this document, not your employer's network, not a site you found online.

The addresses in this lab are internal, private addresses that do not exist on the public internet. The single external address you will see on Day 3 belongs to the scenario, not to you. Do not visit it, scan it, or try to connect to it.

Analysts get fired and prosecuted for scanning things they were not authorised to scan. The habit of checking your authorisation first matters more than any tool in this lab.

3. How to Work Through This Lab

3.1 Time plan

Should take the rest of the afternoon sessions if not, we will use some time in the next day to finish it.

3.2 Set up your workbook before you start

Create one spreadsheet and one document now. You will add to both all week, and having them ready stops you copying things twice.

Where	Name it	What goes in it
Spreadsheet tab 1	All Findings	The 37-row scan export from Section 4.2
Spreadsheet tab 2	Validation	Your five validated findings and verdicts
Spreadsheet tab 3	KEV	The ten CVEs and their KEV status

Where	Name it	What goes in it
Spreadsheet tab 4	Timeline	The Day 3 file system timeline, annotated
Spreadsheet tab 5	IOCs	Your consolidated indicator list
Document 1	MTB Vulnerability Assessment Report	Your Day 2 deliverable
Document 2	MTB Forensic Report - Case MTB-IR-2026-0902	Your Day 3 deliverable

3.3 What a good answer looks like

Most of the marks in this lab are for reasoning, not for reaching the right verdict. Here is the same finding written up three ways.

Standard	Example write-up
Not acceptable	MTB-0016 is a false positive. The version is wrong.
Passing	MTB-0016 (CVE-2017-7494) is a false positive. The CVE affects Samba 3.5.0 up to 4.6.4. MTB-FILE01 runs Samba 4.15.13, which is outside that range.
Strong	MTB-0016 (CVE-2017-7494, SambaCry) is a false positive. NVD states the flaw affects Samba from version 3.5.0 and before 4.6.4, 4.5.10 and 4.4.14. The evidence pack shows MTB-FILE01 running Samba 4.15.13 (smbclient --version, and package samba 2:4.15.13+dfsg-0ubuntu1). 4.15.13 is nine minor versions past the highest affected release, so the host is not vulnerable. The scanner appears to have matched on the presence of a Samba banner rather than on the version. Note this finding IS in the KEV catalog (added 2023-03-30), which is why validating it mattered: unvalidated, it would have been near the top of the remediation queue.

The pattern to copy

Name the finding and the CVE. State the affected range and where you got it. State the detected version and where you got it. State the verdict. Say what the scanner probably did wrong. Note anything that makes the finding interesting beyond the verdict.

Six sentences. Every validated finding in your report should follow that shape.

3.4 Glossary

Terms used in this lab, in the order you will meet them.

Term	What it means
CVE	Common Vulnerabilities and Exposures. A unique identifier for one specific publicly disclosed vulnerability, in the form CVE-2021-41773. The number does not indicate severity.
CVSS	Common Vulnerability Scoring System. A score from 0.0 to 10.0 describing how bad a vulnerability would be if it were exploited. It says nothing about whether anyone is exploiting it.
Severity band	The label attached to a CVSS range: Critical 9.0-10.0, High 7.0-8.9, Medium 4.0-6.9, Low 0.1-3.9, Info 0.0.
Authenticated scan	A scan that logs in to each host with credentials and reads the installed package list, rather than only probing from outside.
False positive	A finding the scanner reported that is not actually present or not actually exploitable on that host.

Term	What it means
True positive	A finding the scanner reported that you have confirmed is genuinely present.
NVD	National Vulnerability Database, run by NIST. The authoritative public record of each CVE, including its description and affected version ranges.
KEV	Known Exploited Vulnerabilities catalog, run by CISA. The authoritative list of vulnerabilities confirmed to be exploited by real attackers in the wild.
Compensating control	A measure that reduces the risk of a vulnerability you cannot fix right now, without fixing the vulnerability itself.
IOC	Indicator of compromise. A concrete, searchable artefact of an intrusion: an IP address, a file path, a file hash, an account name.
Order of volatility	The principle that evidence which disappears fastest must be collected first.
Hash	A fixed-length fingerprint of data. The same input always gives the same hash; any change to the input gives a completely different one.
Webshell	A small script uploaded to a web server that lets an attacker run commands through ordinary web requests.
Beacon	A process on a compromised host that repeatedly connects out to attacker infrastructure, usually on a fixed interval.
Persistence	Anything an attacker installs so their access survives a reboot, a password change, or a process being killed.
Privilege escalation	Moving from a low-privilege account to a higher-privilege one, typically to root or Administrator.
Defanging	Rewriting an address as 185[.]220[.]101[.]47 so it cannot become a clickable link when the IOC list is shared.
Root cause	The specific weakness that allowed the compromise to begin. Not the malware, not the backdoor. The way in.

*Note: All deliverables can be found within this document or through the supplementary spreadsheet linked below:

<https://docs.google.com/spreadsheets/d/1L2qj3zsDwjoUzXCm-GJFfiw4ZkgCFkfzz8nOaTP6vRw/edit?usp=sharing>

4. Day 1, Part 1: Triage the Scan

An authenticated vulnerability scan of the five hosts was run on 8 September 2026. It ran for 41 minutes and produced 37 findings. That export is printed in full below.

4.1 What an authenticated scan is, and why it matters

An unauthenticated scan looks at a host from the outside: which ports are open, what banners the services return. An authenticated scan logs in with credentials and reads the actual installed package list.

Authenticated scanning finds far more, because most vulnerabilities are in software that never exposes a banner. It also produces different kinds of false positives, which is exactly what you will be hunting for in Part 2.

4.2 The scan export

Findings are listed in scanner order, grouped by host. The second line of each vulnerability cell is the service and version the scanner detected. You will need those versions in Part 2.

ID	Host	Port	Severity	CVSS	CVE	Vulnerability / Detected version
MTB-0001	MTB-WEB01	80/tcp	Critical	9.8	CVE-2021-42013	Apache HTTP Server Path Traversal and RCE <i>Apache httpd 2.4.49</i>
MTB-0002	MTB-WEB01	80/tcp	High	7.5	CVE-2021-41773	Apache HTTP Server Path Traversal and RCE <i>Apache httpd 2.4.49</i>
MTB-0003	MTB-WEB01	general	High	7.8	CVE-2021-4034	polkit pkexec Local Privilege Escalation (PwnKit) <i>policykit-1 0.105-31</i>
MTB-0004	MTB-WEB01	80/tcp	Medium	6.1	CVE-2020-11022	jQuery Cross Site Scripting <i>jQuery 3.4.1</i>
MTB-0005	MTB-WEB01	80/tcp	Medium	5.3	-	ServerTokens Full Information Disclosure <i>Apache httpd 2.4.49</i>
MTB-0006	MTB-WEB01	80/tcp	Low	3.7	-	Web Server Directory Listing Enabled <i>Apache httpd 2.4.49</i>
MTB-0007	MTB-WEB01	22/tcp	Low	3.7	-	SSH Weak MAC Algorithms Supported <i>OpenSSH 8.9p1</i>
MTB-0008	MTB-WEB01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS <i>Linux kernel 5.15</i>
MTB-0009	MTB-APP01	21/tcp	Critical	9.8	CVE-2011-2523	vsftpd Backdoor Command Execution <i>vsftpd 2.3.4</i>
MTB-0010	MTB-APP01	21/tcp	High	7.5	-	FTP Anonymous Login Permitted <i>vsftpd 2.3.4</i>
MTB-0011	MTB-APP01	21/tcp	High	7.4	-	FTP Credentials Transmitted in Cleartext <i>vsftpd 2.3.4</i>
MTB-0012	MTB-APP01	80/tcp	Critical	9.8	CVE-2014-6271	GNU Bash Shellshock Remote Code Execution <i>GNU bash 5.1.16</i>
MTB-0013	MTB-APP01	22/tcp	Medium	5.3	-	SSH PasswordAuthentication Enabled <i>OpenSSH 8.9p1</i>
MTB-0014	MTB-APP01	general	Medium	6.5	CVE-2023-4911	glibc ld.so Buffer Overflow (Looney Tunables) <i>glibc 2.35</i>
MTB-0015	MTB-APP01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS <i>Linux kernel 5.15</i>
MTB-0016	MTB-FILE01	445/tcp	Critical	9.8	CVE-2017-7494	Samba is_known_pipename RCE (SambaCry) <i>Samba 4.15.13</i>
MTB-0017	MTB-FILE01	23/tcp	High	7.5	-	Telnet Service Detected - Cleartext Protocol <i>telnetd</i>
MTB-0018	MTB-FILE01	445/tcp	High	8.1	-	SMB Signing Not Required <i>Samba 4.15.13</i>
MTB-0019	MTB-FILE01	general	High	7.8	CVE-2021-4034	polkit pkexec Local Privilege Escalation (PwnKit) <i>policykit-1 0.105-31</i>
MTB-0020	MTB-FILE01	445/tcp	Medium	5.0	-	SMB Null Session Enumeration Possible <i>Samba 4.15.13</i>
MTB-0021	MTB-FILE01	22/tcp	Medium	5.3	-	SSH PermitRootLogin Enabled <i>OpenSSH 8.9p1</i>

ID	Host	Port	Severity	CVSS	CVE	Vulnerability / Detected version
MTB-0022	MTB-FILE01	general	Low	2.4	-	World-Writable Files Present in /opt <i>Filesystem</i>
MTB-0023	MTB-FILE01	general	Info	0.0	-	OS Detection: Ubuntu 22.04 LTS <i>Linux kernel 5.15</i>
MTB-0024	MTB-DC01	3389/tcp	Critical	9.8	CVE-2019-0708	Remote Desktop Services RCE (BlueKeep) <i>Windows Server 2019</i>
MTB-0025	MTB-DC01	445/tcp	High	8.8	CVE-2020-1472	Netlogon Elevation of Privilege (ZeroLogon) <i>Windows Server 2019</i>
MTB-0026	MTB-DC01	3389/tcp	Medium	5.9	-	RDP Weak Encryption Level Configured <i>Windows Server 2019</i>
MTB-0027	MTB-DC01	445/tcp	Medium	5.3	-	SMBv1 Protocol Enabled <i>Windows Server 2019</i>
MTB-0028	MTB-DC01	general	Medium	6.5	-	Local Administrator Password Never Expires <i>Windows Server 2019</i>
MTB-0029	MTB-DC01	general	Low	3.3	-	Defender Real-Time Protection Disabled <i>Windows Server 2019</i>
MTB-0030	MTB-DC01	general	Info	0.0	-	OS Detection: Windows Server 2019 <i>Windows Server 2019</i>
MTB-0031	MTB-DVWA01	80/tcp	Critical	9.8	-	SQL Injection in Application Parameter <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0032	MTB-DVWA01	80/tcp	High	8.8	-	OS Command Injection in Application Parameter <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0033	MTB-DVWA01	80/tcp	High	7.5	-	Unrestricted File Upload Permitted <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0034	MTB-DVWA01	80/tcp	Medium	6.1	-	Reflected Cross Site Scripting <i>Apache 2.4.52 / PHP 8.1</i>
MTB-0035	MTB-DVWA01	80/tcp	Medium	5.3	-	Session Cookie Missing HttpOnly and Secure <i>PHP 8.1</i>
MTB-0036	MTB-DVWA01	80/tcp	Medium	4.3	-	Login Form Served Over Plain HTTP <i>Apache 2.4.52</i>
MTB-0037	MTB-DVWA01	80/tcp	Low	3.1	-	HTTP TRACE Method Enabled <i>Apache 2.4.52</i>

4.3 Step by step

1. Open a new spreadsheet. Create a tab called **All Findings** and copy in the seven columns above. You will reuse this tab all through Days 1 and 2.
2. Count the findings by severity. Fill in the table in Section 4.4. The five counts must add up to 37 - if they do not, you have missed a row.
3. Sort your sheet by CVSS score, highest first. Record the ten highest-severity findings in the table in Section 4.5, along with the host each one affects.
4. Look at your top ten and ask a harder question: does anything here look wrong? You know what these hosts run. Note any finding you suspect the scanner got wrong, and why you suspect it. You are not proving anything yet - you are forming a hypothesis you will test in Part 2.

Why the ranking you just produced is not good enough

You have ranked 37 findings by a number the scanner gave you. Two things are wrong with that ranking, and you will fix them over the next two sessions.

First, some of these findings are not real. Section 5 makes you prove which.

Second, CVSS measures how bad a vulnerability would be if exploited. It says nothing about whether anyone is actually exploiting it. Section 6 fixes that.

4.4 Record: severity breakdown

Severity	Count	Your notes
Critical	6	Mostly RCEs, SQL Injection
High	10	Privilege Escalations, Command Injections
Medium	12	Weak Encryption, Legacy SMB and SSH protocols
Low	5	Misconfiguration, Defender disabled,
Info	4	OS Detection
TOTAL	37	

4.5 Record: top ten by CVSS

Rank	Finding ID	Host	CVSS	Vulnerability	Suspicious? Why?
1	MTB-0001	MTB-WEB01	9.8	Apache HTTP Server Path Traversal and RCE (CVE-2021-42013)	No, typical unpatched server vulnerability
2	MTB-0009	MTB-APP01	9.8	vsftpd Backdoor Command Execution (CVE-2011-2523)	Yes, this backdoor command execution is rarely seen in modern systems
3	MTB-0012	MTB-APP01	9.8	GNU Bash Shellshock Remote Code Execution (CVE-2014-6271)	Yes, likely a false positive. Ubuntu 22.04 with Bash 5.1.16 is not vulnerable to a 2014 exploit

Rank	Finding ID	Host	CVSS	Vulnerability	Suspicious? Why?
4	MTB-0016	MTB-FILE 01	9.8	Samba is_known_pipename RCE (SambaCry) (CVE-2017-7494)	Yes, likely a false positive. Samba 4.15.13 is a much newer version than the one affected by 2017 SambaCry
5	MTB-0024	MTB-DC01	9.8	Remote Desktop Services RCE (BlueKeep) (CVE-2019-0708)	Yes, Windows Server 2019 is not vulnerable to BlueKeep, which primarily affected Windows Server 2008 R2 and older
6	MTB-0031	MTB-DVW A01	9.8	SQL Injection in Application Parameter	No, standard web application vulnerability
7	MTB-0025	MTB-DC01	8.8	Netlogon Elevation of Privilege (ZeroLogon) (CVE-2020-1472)	No, typical missing security patch on a domain controller.
8	MTB-0032	MTB-DVW A01	8.8	OS Command Injection in Application Parameter	No, standard web application vulnerability
9	MTB-0018	MTB-FILE 01	8.1	SMB Signing Not Required	No, common legacy misconfiguration in internal networks
10	MTB-0003	MTB-WEB 01	7.8	polkit pkexec Local Privilege Escalation (PwnKit) (CVE-2021-4034)	No, typical unpatched local privilege escalation

5. Day 1, Part 2: Validate the Findings

A scanner does not know things. It matches patterns. When it sees a service banner or a package name, it compares that against a signature and reports whatever the signature says. Signatures are written by people, and people write them loosely.

The result is that a meaningful share of every scan is wrong. Your job as an analyst is to find out which parts, before the infrastructure team spends a weekend patching something that was never vulnerable.

5.1 How analysts validate a finding

You do not need to re-scan anything, and in a real bank you usually cannot. Validation is a comparison:

- **The detected version** - what the scanner says is actually installed on the host. This is in the second line of each vulnerability cell in Section 4.2.
- **The affected version range** - what the vulnerability actually applies to, according to the authoritative source. This is on NVD.

If the detected version falls inside the affected range, the finding is a true positive. If it falls outside, the finding is a false positive and you can say so with evidence.

This is the single most valuable skill in this lab

Anyone can forward a scan report. An analyst is someone who can look at a Critical 9.8 finding and say, with evidence, that it does not apply to this host.

That judgement is what stops a security team from burning its credibility on findings that were never real.

5.2 Looking up a CVE on NVD

1. Go to nvd.nist.gov in your browser.
2. Paste the CVE identifier into the search box at the top of the page and press Enter. You can also go straight to the record by typing the address nvd.nist.gov/vuln/detail/CVE-2021-41773 and swapping in whichever CVE you want.
3. On the CVE page, read the **Description**. For most of the CVEs in this lab the description states the affected versions in plain English, in the last sentence. Read that sentence carefully - it is usually the whole answer.
4. Compare the version named in the description against the detected version from Section 4.2. Write down your conclusion and quote the exact wording you based it on.
5. Optional. Further down the page there is a section called **Known Affected Software Configurations**, which lists the formal version ranges in the form **From (including) 3.5.0 and Up to (excluding) 4.6.4**. Use it to confirm what the description already told you.

Use the description. Do not wait on the version tables.

The Known Affected Software Configurations section is built by a script that runs after the page opens. Until it finishes it displays the words "CPEs loading, please wait." On older CVEs with hundreds of matching product entries it can take a long time, and it sometimes never finishes at all.

That is expected behaviour, not a broken page and not something you did wrong. Do not sit and wait for it.

The plain-English description at the top of the page answers the question in one sentence, and it is the same information. Every CVE in this lab names its affected versions there. Read it, record it, move on.

If you want the formal ranges and the table will not load, Appendix A.1 prints them.

5.3 The validation evidence pack

Your team lead pulled the following detail from the scan logs and from the hosts themselves. This is your supporting evidence. Read it alongside the NVD ranges.

MTB-WEB01 (10.50.2.24)

```
$ curl -sI http://10.50.2.24/
HTTP/1.1 200 OK
Server: Apache/2.4.49 (Ubuntu)

$ dpkg -l | grep -E 'apache2 |policykit'
ii apache2                2.4.49-1ubuntu1      amd64 Apache HTTP Server
ii policykit-1            0.105-31             amd64 authorization framework

$ grep -A2 'ScriptAlias' /etc/apache2/sites-enabled/000-default.conf
ScriptAlias /cgi-bin/ /usr/lib/cgi-bin/
<Directory />
    Require all granted
```

MTB-APP01 (10.50.2.25)

```
$ nc 10.50.2.25 21
220 (vsFTPd 2.3.4)

$ bash --version
GNU bash, version 5.1.16(1)-release (x86_64-pc-linux-gnu)

$ dpkg -l | grep -E 'bash |libc6 '
ii bash                    5.1-6ubuntu1         amd64 GNU Bourne Again SHell
ii libc6                   2.35-0ubuntu3.4      amd64 GNU C Library
```

MTB-FILE01 (10.50.2.26)

```
$ smbclient --version
Version 4.15.13-Ubuntu

$ dpkg -l | grep -E 'samba |policykit'
ii samba                   2:4.15.13+dfsg-0ubuntu1 amd64 SMB/CIFS file server
ii policykit-1            0.105-31             amd64 authorization framework
```

MTB-DC01 (10.50.2.10)

```
PS> Get-ComputerInfo | Select OsName, OsVersion, OsBuildNumber

OsName       : Microsoft Windows Server 2019 Standard
OsVersion    : 10.0.17763
OsBuildNumber : 17763

PS> Get-HotFix | Measure-Object
Count : 4          # last cumulative update applied 2020-06-09
```

5.4 Step by step

1. Pick five findings from your top ten. Choose a mix of severities, and make sure you include any finding you flagged as suspicious in Section 4.3.
2. For each one, look up the CVE on NVD using the method in Section 5.2. Findings with no CVE are configuration weaknesses - validate those against the evidence pack in Section 5.3 instead.
3. Write down the affected version range, the detected version, and your verdict. Record all of this in the table in Section 5.6.
4. Identify at least one finding you can prove is a false positive. Explain specifically why the scanner was wrong, quoting the version range and the detected version. "It looks wrong" is not an answer; "the CVE affects Samba 3.5.0 up to 4.6.4 and this host runs 4.15.13" is.
5. Identify at least one finding you can prove is a true positive, with the exact evidence that confirms it.
6. Adjust your severity ranking if validation changed your view of any finding's real risk. Note every change and why you made it.

There is more than one false positive in this scan

The lab asks you to find at least one. There are three.

Two of them are sitting in your top ten, because both were reported as Critical 9.8. That is the point: the findings most likely to be wrong are often the ones screaming loudest for attention.

If you find all three with correct reasoning, say so in your report.

5.5 Where to look first

If you are stuck, these three questions will each lead you to something:

- One host is running a Linux service whose version number is far higher than any version the CVE was ever reported against. Which host, and which service?
- One CVE affects a Windows operating system generation that is older than the one MTB-DC01 is running. Read the NVD description for the RDP finding and compare it against the PowerShell output in Section 5.3.
- One finding names a piece of software that is a full major version ahead of anything the vulnerability applies to. The evidence pack prints its exact version.

5.6 Record: validated findings

Finding ID	CVE	Detected version	Affected range (from NVD)	Verdict	Evidence and reasoning
MTB-0009	CVE-2011-2523	vsftpd 2.3.4	2.3.4	True Positive	Confirmed service vsftpd 2.3.4 is actively running on port 21 which matches the affected range

Finding ID	CVE	Detected version	Affected range (from NVD)	Verdict	Evidence and reasoning
MTB-0012	CVE-2014-6271	GNU bash 5.1.16	Up to 4.3	False Positive	Confirmed service GNU bash 5.1.16 falls out of affected version range
MTB-0016	CVE-2017-7494	Samba 4.15.13	Version 3.5.0 and before 4.6.4, 4.5.10 and 4.4.14	False Positive	Confirmed service Samba 4.15.13 falls out of range of affected version range
MTB-0002	CVE-2021-41773	Apache httpd 2.4.49	2.4.49	True Positive	Confirmed service Apache httpd 2.4.49 is actively running which matches the affected range allowing server path traversal/RCE
MTB-0003	CVE-2021-4034	policykit-1 0.105-31	Up to 0.120	True Positive	Confirmed service policykit-1 version 0.105-31 is actively running falling in the affected range allowing for local privilege escalation

Give each row several lines in your own document. The reasoning column is where the marks are.

6. Day 2, Part 1: Threat Intelligence Cross-Reference

You now have a list of findings you believe are real. They are still ranked by CVSS, which is a severity score, not a risk score.

CVSS answers: if someone exploited this, how bad would it be? It does not answer: is anyone actually exploiting this? Those are different questions, and a bank has to answer the second one to decide what to fix first.

6.1 What the KEV catalog is

The Cybersecurity and Infrastructure Security Agency maintains the Known Exploited Vulnerabilities catalog: the authoritative list of vulnerabilities confirmed to be exploited in the wild by real attackers.

Under Binding Operational Directive 22-01, US federal civilian agencies are required to remediate KEV-listed vulnerabilities on a fixed deadline. Banks are not bound by that directive, but nearly every large financial institution uses KEV the same way, because it is the single most actionable prioritisation signal available and it is free.

A vulnerability being in KEV means somebody has actually used it against a real organisation. A vulnerability not being in KEV does not mean it is safe - it means there is no confirmed record of exploitation. That distinction matters when you write your report.

6.2 Three ways to check KEV status, in order of reliability

You do not have to browse the catalog. The KEV table on the CISA site is paginated at twenty entries per page across roughly seventy pages, and its filters are not always dependable. Use method one. The other two are there so you always have a way through.

Method 1 - the KEV panel on the NVD page (recommended)

You already have every CVE page open from Section 5. NVD prints the KEV status directly on it, so this costs you no extra page loads.

1. On the CVE page at nvd.nist.gov/vuln/detail/CVE-2017-7494, scroll down past the References table.
2. If the CVE is in KEV, you will see a panel headed **This CVE is in CISA's Known Exploited Vulnerabilities Catalog**, containing four columns: Vulnerability Name, Date Added, Due Date and Required Action.
3. If that panel is not on the page, the CVE is not in KEV. An absent panel is a real result. Record it as such.
4. Record the Date Added and the Due Date.

Method 2 - download the catalog and search it (most reliable)

This is how analysts actually do it, because it works offline and it scales to hundreds of CVEs.

Go to cisa.gov/known-exploited-vulnerabilities-catalog.

Under the paragraph headed **How to use the KEV Catalog**, there is a line reading "The KEV catalog is also available in these formats" followed by links marked **CSV** and **JSON**. Click **CSV**.

Open the downloaded file in Excel, Google Sheets or LibreOffice Calc. It has roughly 1,700 rows, one per vulnerability.

Press Ctrl+F (Cmd+F on a Mac) and search for each CVE identifier. Found means in KEV. Not found means not in KEV.

The columns you need are `cveID`, `dateAdded`, `dueDate` and `knownRansomwareCampaignUse`.

Method 3 - the search box on the CISA page

The catalog page has a search field above the results table. It works, but the page is long and heavily filtered, so if a search returns nothing, confirm with method 1 or 2 before concluding the CVE is absent.

Two date formats, same dates

NVD writes dates American style as MM/DD/YYYY, so 30 March 2023 appears as 03/30/2023.

The CISA catalog and its CSV write dates ISO style as YYYY-MM-DD, so the same day appears as 2023-03-30.

These are the same date. Pick one format for your report and convert everything into it, so your timeline is not ambiguous to whoever reads it.

Check the false positives too

Do not skip the CVEs you disproved in Section 5. Look them up in KEV anyway.

You will find that all three false positives are in the KEV catalog. That is the finding, and it belongs in your report.

A team that had not validated would have put all three at the top of the remediation queue, on the strength of a Critical score plus confirmed in-the-wild exploitation. They would have spent the week patching three things that were never vulnerable, while the finding that actually mattered sat further down the list.

6.3 The ten CVEs in this assessment

CVE	CVSS	In KEV?	Date added	Ransomware ?	Your priority notes
CVE-2021-42013	9.8	Yes	November 03, 2021	Yes	High priority, actively exploited path traversal/RCE
CVE-2021-41773	7.5	Yes	November 03, 2021	Unknown	High priority, Confirmed true positive based on the Apache server path traversal/RCE
CVE-2021-4034	7.8	Yes	June 27, 2022	Unknown	High Priority, PwnKit local privilege escalation
CVE-2020-11022	6.1	No		Unknown	Medium priority, jQuery XSS
CVE-2011-2523	9.8	No		Unknown	High priority, true positive, vsftpd backdoor allowing command execution
CVE-2014-6271	9.8	Yes	January 28, 2022	Unknown	Low priority, confirmed false positive in previous steps, as Bash 5.1.16 version is too new to be vulnerable to Shellshock
CVE-2017-7494	9.8	Yes	March 30, 2023	Yes	Low priority, confirmed false positive in previous steps, as Samba 4.15.13 is newer than the vulnerable SambaCry range
CVE-2023-4911	6.5	Yes	November 21, 2023	Unknown	Buffer overflow, Base glibc version matches
CVE-2019-0708	9.8	Yes	November 03, 2021	Yes	Low priority, confirmed false positive as Windows Server 2019 is not vulnerable to BlueKeep
CVE-2020-1472	8.8	Yes	November 03, 2021	Yes	High priority, Zerologon privilege escalation on a Domain Controller

*Highlighted CVEs are false positives within our systems

6.4 Step by step

1. Check all ten CVEs against KEV and fill in the table above.
2. Flag every validated finding that appears in KEV as actively exploited, regardless of its CVSS score.
3. Re-prioritise your findings list so that confirmed real-world exploitation outweighs CVSS score alone. Remove the false positives from the list entirely - they are not findings.
4. Find at least one case where this reordering moves a finding down. Explain in writing what changed and why. There is a clear one in this data set: a Critical 9.8 finding that is not in KEV, which should now rank below two findings scored lower than it.
5. For your single highest-priority finding, record the CVE publication date and the KEV addition date. Note the gap between them and write two sentences on what that gap tells you about how fast a newly disclosed vulnerability becomes a real-world threat.

6.5 Record: re-prioritised findings

New rank	Finding ID	Host	CVSS	In KEV	Why it sits here
1	MTB-0001	MTB-WEB01	9.8	Yes	Highest CVSS among KEV findings, confirmed RCE
2	MTB-0025	MTB-DC01	8.8	Yes	High CVSS in KEV findings, Domain Controller risk
3	MTB-0003	MTB-WEB01	7.8	Yes	In KEV, High CVSS privilege escalation
4	MTB-0019	MTB-FILE01	7.8	Yes	In KEV, High CVSS privilege escalation
5	MTB-0002	MTB-WEB01	7.5	Yes	In KEV, Can lead to RCE
6	MTB-0014	MTB-APP01	6.5	Yes	In KEV, Prioritized over non-KEV findings despite lower CVSS
7	MTB-0009	MTB-APP01	9.8	No	Deprioritized despite 9.8 CVSS because it is not in KEV
8	MTB-0004	MTB-WEB01	6.1	No	Lowest CVSS among non-KEV findings

Reordering of MTB-0009:

Finding MTB-0009 (CVE-2011-2523) has a Critical 9.8 CVSS score but is not in the KEV catalog. Because this framework dictates that real-world active exploitation outranks theoretical severity, it now correctly sits at rank #7 below lower-scored findings like MTB-0014 (6.5), which are actively being exploited by attackers.

Highest Priority Finding:

The highest priority finding was MTB-0001 (CVE-2021-42013). The vulnerability was published to the NVD on October 8, 2021, and added to the KEV catalog on November 3, 2021. This gap of 27 days shows just how fast threat actors can capitalize on vulnerabilities, turning them into real world exploits.

7. Day 2, Part 2: Assessment Report and Remediation Plan

Everything so far has been analysis. This section turns it into something the infrastructure team can act on. A report that lists problems without telling anyone what to do about them is not a deliverable.

7.1 What goes in the report

1. List your validated, threat-intelligence-adjusted findings in priority order. Each one needs the affected host, the evidence that confirms it, and a specific recommended fix. "Patch it" is not a fix; "upgrade apache2 to 2.4.51 or later" is.
2. For any finding that cannot be patched immediately, propose a compensating control instead of leaving it open. Examples: network segmentation, disabling the vulnerable feature or module, a tighter firewall rule, removing a service that is no longer needed.
3. Include a section on the false positives. Name each one, state what the scanner claimed, state what you proved, and state the evidence. This section is what tells your reader the rest of the report can be trusted.
4. Recommend a recurring scan cadence for this environment going forward, and justify the frequency you picked. Different hosts may deserve different cadences - MTB-WEB01 is internet facing and MTB-DVWA01 is not.
5. Write a short executive summary, three to five sentences, for a non-technical stakeholder. No CVE numbers, no CVSS scores. What is the risk to the bank, what are you doing about it, and by when.

7.2 Compensating controls: worked example

Suppose MTB-FILE01 cannot have Telnet removed this month because a legacy backup job still uses it. You cannot fix the finding, so you reduce what an attacker could do with it:

- Restrict the Telnet port to the single management subnet with a firewall rule, so it is not reachable from user or guest zones.
- Alert on any successful Telnet authentication from outside that subnet, so an attempt is visible even if it succeeds.
- Record a dated commitment to migrate the backup job to SSH, with a named owner, so the control does not quietly become permanent.

That is a compensating control: the vulnerability is still there, the exposure is smaller, the risk is being tracked rather than forgotten.

Write the executive summary last, and write it for a person

The stakeholder reading it does not know what CVSS is and does not need to. They need to know whether the bank is exposed, how badly, and what happens next.

A good test: if you read your summary aloud to someone outside IT and they can repeat back what the risk is and what you are doing about it, it works.

7.3 Report structure to follow

Section	Contents
Executive summary	Three to five sentences, plain language, no jargon.
Scope and method	Which hosts, when the scan ran, what kind of scan, how you validated.
Prioritised findings	Your re-prioritised list. Host, evidence, fix, and why it sits where it does.
False positives	Each disproved finding, what the scanner claimed, and your evidence.
Compensating controls	For anything that cannot be patched now, with a named owner and a date.
Recommended cadence	Your proposed scan frequency, with justification.

Meridian Trust Bank: Assessment Report and Remediation Plan

Executive Summary:

This security assessment identified critical vulnerabilities within the Meridian Bank's infrastructure that could allow threat actors to take control of our public-facing web servers and internal network controllers. We have prioritized these risks based on current, real-world attacker behavior and are deploying software updates to our exposed systems. For a legacy internal system that cannot be patched instantly, we are implementing strict network firewalls to block unauthorized access. All critical updates and defensive controls will be enforced as soon as possible to ensure the bank's data and operations remain secure.

Scope and Method:

Hosts Scanned: MTB-WEB01, MTB-APP01, MTB-FILE01, MTB-DC01, MTB-DVWA01

Scan Date: September 2026

Scan Type: Authenticated network and web application vulnerability scan

Validation: Findings were manually verified by cross referencing with the NIST CVE database and KEV catalog

Prioritized Findings:

1. MTB-0001 (Host: MTB-WEB01):

Evidence: Confirmed Apache 2.4.49 is installed, and configuration review confirmed the cgi-bin directory is enabled with Require all granted.

Fix: Upgrade the apache2 package to version 2.4.51 or later.

Why it sits here: It is a Critical (9.8) RCE vulnerability actively exploited in the wild (in the CISA KEV catalog).

2. MTB-0025 (Host: MTB-DC01):

Evidence: Scanner successfully identified missing Netlogon security patches.

Fix: Apply the Microsoft Security Update addressing CVE-2020-1472.

Why it sits here: Actively exploited (in KEV) and poses a total compromise risk to the domain controller.

3. MTB-0003 (Host: MTB-WEB01):

Evidence: dpkg confirmed policykit-1 version 0.105-31 is installed.

Fix: Upgrade policykit-1 to the vendor-patched release (0.105-31ubuntu0.1 or later).

Why it sits here: Actively exploited (in KEV) local privilege escalation on an internet-facing host.

4. MTB-0019 (Host: MTB-FILE01):

Evidence: dpkg confirmed policykit-1 version 0.105-31 is installed.

Fix: Upgrade policykit-1 to the vendor-patched release.

Why it sits here: Same risk as #3, but ranked lower as the host is internal rather than internet-facing.

5. MTB-0002 (Host: MTB-WEB01):

Evidence: Same configuration evidence as MTB-0001.

Fix: Upgrade the apache2 package to version 2.4.51 or later.

Why it sits here: In KEV, but acts as a precursor/directory traversal component of the RCE.

6. MTB-0014 (Host: MTB-APP01):

Evidence: dpkg confirmed libc6 version 2.35 is installed without the backported patch.

Fix: Upgrade libc6 to the patched Ubuntu release addressing CVE-2023-4911.

Why it sits here: In KEV, but has a lower CVSS score (6.5) than the preceding vulnerabilities.

7. MTB-0009 (Host: MTB-APP01):

Evidence: nc banner grabbing confirmed vsFTPD 2.3.4 is actively running.

Fix: Uninstall vsftpd and migrate to a secure file transfer protocol like SFTP via OpenSSH.

Why it sits here: Despite a 9.8 CVSS, it is a legacy vulnerability not found in the CISA KEV catalog, dropping its real-world priority.

8. MTB-0004 (Host: MTB-WEB01):

Evidence: Scanner identified jQuery 3.4.1 in use by the web application.

Fix: Upgrade jQuery library to version 3.5.0 or later.

Why it sits here: Not actively exploited (Not in KEV) and carries the lowest severity score (6.1) of the valid findings.

False Positives:

MTB-0012 (GNU Bash Shellshock): Findings show that Bash was vulnerable to RCE. Manual verification showed that the host runs version 5.1.16 which is far newer than the vulnerable range (4.3 or less).

MTB-0016 (SambaCry RCE): The scanner claimed the Samba service was vulnerable. Verification proved version 4.15.13 is installed, past the vulnerable range.

MTB-0024 (BlueKeep RCE): The scanner flagged RDP as vulnerable. Verification confirmed the OS is Windows Server 2019, which is not vulnerable to this legacy exploit.

Compensating Controls:

Finding: MTB-0009 (vsFTPD Backdoor on MTB-APP01)

Cannot be immediately removed because a legacy automated reporting job relies on FTP for file transfer

Control: Implement a host-based firewall rule to restrict Port 21 access exclusively to the IP address of the legacy reporting server. Configure SIEM alerts for any failed or successful FTP authentication attempts from outside that specific IP.

Owner: John Doe

Migration Date: October 15, 2026 (Service will be moved to SFTP)

Recommended Cadence:

Internet-Facing Hosts (MTB-WEB01, MTB-DVWA01): Weekly authenticated vulnerability scans

Justification: These systems have direct exposure to the public internet, meaning the time-to-exploitation for newly disclosed vulnerabilities is extremely short.

Internal Infrastructure (MTB-APP01, MTB-FILE01, MTB-DC01): Monthly authenticated vulnerability scans.

Justification: Balances infrastructure team workload and network overhead while maintaining adequate visibility into lateral movement and privilege escalation risks.

8. Day 3, Part 1: The Alert and Volatile Evidence

Six days after your assessment, one of the hosts you looked at triggered a security monitoring alert. Your team lead believes it has actually been compromised, most likely through one of the vulnerabilities you flagged.

You have been assigned to investigate: preserve the evidence properly, determine exactly what happened, and produce a report that would hold up if a regulator asked to see it.

8.1 The alert

```

MERIDIAN TRUST BANK - SECURITY OPERATIONS CENTRE
ALERT ID      : SOC-2026-09-02-0416
RAISED       : 2026-09-02 09:47:12 UTC
SEVERITY      : High
ASSET        : MTB-WEB01 (10.50.2.24) - DMZ customer portal
RULE         : Periodic outbound connection to non-allowlisted destination

DETAIL        : Host has established an outbound TCP session to
                185.220.101.47 on port 8443 at a consistent 5 minute
                interval since approximately 02:31 UTC. Destination is
                not present in any approved egress allowlist.

                A root-owned process named 'apache-worker' is running.
                This process name does not appear in the Meridian Trust
                standard Apache build manifest.

ACTION       : Host isolated by network ACL at 10:05 UTC.
                HOST LEFT POWERED ON at the request of the IR team.
    
```

8.2 Order of volatility

Evidence disappears at different speeds. Some of it is gone the moment you pull the power; some of it will still be there in a year. The order in which you collect evidence therefore matters enormously, and getting it wrong destroys information you can never get back.

Order	Evidence type	Lifetime	Examples
1	CPU registers and cache	Nanoseconds	Not practically collectable
2	Memory contents	Until power off	Running processes, open sockets, decrypted data, injected code
3	Network state	Seconds to minutes	Active connections, listening ports, ARP and routing tables
4	Running system state	Until reboot	Logged-in users, open files, loaded modules, running services
5	Disk contents	Until overwritten	Files, logs, deleted data still in unallocated space
6	Remote and archived logs	Weeks to years	SIEM records, backups, off-host log copies

This is why the host was left powered on

The IR team's instruction to leave MTB-WEB01 running was deliberate. Pulling the power would have destroyed everything in categories 2, 3 and 4 instantly and permanently.

The running process, the established outbound session, and the open file handles that tie them together only exist while the machine is on. Imaging the disk first and then powering off would have produced a technically perfect disk image of an incident you could no longer fully explain.

Note that collecting volatile evidence does change the system slightly - your own commands create processes and write entries. That is accepted practice. You document what you ran and when, so the change is on the record rather than hidden.

8.3 The volatile capture

The on-call responder captured the following from MTB-WEB01 while it was still running, before anything was imaged. Each capture is timestamped. Read them in the order they were taken.

Capture 1 - running processes, 10:12:41 UTC

```
$ ps -ef
UID      PID  PPID  C  STIME TTY      TIME    CMD
root      1      0  0 Sep01 ?        00:00:04 /sbin/init
root     812      1  0 Sep01 ?        00:00:00 /usr/sbin/cron -f
root     844      1  0 Sep01 ?        00:00:01 /usr/sbin/sshd -D
root     901      1  0 Sep01 ?        00:00:02 /usr/sbin/apache2 -k start
www-data 903     901  0 Sep01 ?        00:00:00 /usr/sbin/apache2 -k start
www-data 904     901  0 Sep01 ?        00:00:00 /usr/sbin/apache2 -k start
root    1244      1  0 02:30 ?        00:00:00 /bin/bash /usr/local/bin/apache-worker
root    1247    1244  0 02:30 ?        00:00:00 sleep 300
mtbadmin 1401    844  0 09:47 ?        00:00:00 sshd: mtbadmin@pts/0
```

Capture 2 - network connections, 10:13:02 UTC

```
$ ss -tunap
Netid State  Local Address:Port    Peer Address:Port  Process
tcp    LISTEN  0.0.0.0:22        0.0.0.0:*          users:(("sshd",pid=844))
tcp    LISTEN  0.0.0.0:80        0.0.0.0:*          users:(("apache2",pid=901))
tcp    ESTAB   10.50.2.24:49882  185.220.101.47:8443 users:(("apache-worker",pid=1244))
tcp    ESTAB   10.50.2.24:22     10.50.1.31:50110   users:(("sshd",pid=1401))
```

Capture 3 - logged-in users and login history, 10:13:20 UTC

```
$ who
mtbadmin pts/0          2026-09-02 09:47 (10.50.1.31)

$ last -n 8
mtbadmin pts/0      10.50.1.31      Wed Sep  2 09:47    still logged in
root      pts/1      185.220.101.47   Wed Sep  2 02:24 - 02:44 (00:20)
mtbadmin pts/0      10.50.1.31      Tue Sep  1 09:14 - 11:52 (02:38)
reboot    system boot 5.15.0-91      Tue May 19 03:08    still running
```

Capture 4 - open network files, 10:13:44 UTC

```
$ lsof -i -n -P
COMMAND      PID    USER   FD  TYPE  NODE  NAME
sshd          844    root    3u   IPv4  TCP   *:22 (LISTEN)
apache2       901    root    4u   IPv4  TCP   *:80 (LISTEN)
apache-worker 1244    root    3u   IPv4  TCP   10.50.2.24:49882->185.220.101.47:8443 (ESTABLISHED)
sshd          1401    root    3u   IPv4  TCP   10.50.2.24:22->10.50.1.31:50110 (ESTABLISHED)
```

Capture 5 - scheduled tasks, 10:14:05 UTC

```
$ cat /etc/cron.d/*
# Apache log maintenance
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
*/5 * * * * root /usr/local/bin/apache-worker >/dev/null 2>&1
```

8.4 Pulling indicators out with CyberChef

You could read through the captures by hand and copy addresses into a list. On a real incident with megabytes of logs you cannot. CyberChef does it in one step, and you will use the same technique on the much larger log files in Section 11.

1. Open gchq.github.io/CyberChef in your browser. The screen has four panels: **Operations** on the left, **Recipe** next to it, **Input** top right, **Output** bottom right.
2. In the Operations search box at the top left, type **Extract IP**. The operation **Extract IP addresses** appears under the **Extractors** category.
3. Drag **Extract IP addresses** from Operations into the Recipe panel. You can also double-click it.
4. Copy the text of Captures 1 to 5 above and paste it into the **Input** panel.
5. The Output panel updates immediately with every IP address found. Tick the **Unique** checkbox in the operation's settings to collapse duplicates.
6. Add a second operation: search for **Sort**, drag **Sort** below Extract IP addresses in the recipe. Your output is now a clean, deduplicated, sorted address list.

Recipe

Extract IP addresses

☒ IPv4 ☐ IPv6 ☐ Remove local IPv4 addresses

☐ Display total ☐ Sort ☒ Unique

Sort

Delimiter: Line feed ☐ Reverse Order: Alphabetical (case...)

Input

```

mtbadmin pts/0 10.50.1.31 Wed Sep 2 09:47 still logged in
root pts/1 185.220.101.47 Wed Sep 2 02:24 - 02:44 (00:20)
mtbadmin pts/0 10.50.1.31 Tue Sep 1 09:14 - 11:52 (02:38)
reboot system boot 5.15.0-91 Tue May 19 03:08 still running

$ lsof -i -n -P
COMMAND PID USER FD TYPE NODE NAME
sshd 844 root 3u IPv4 TCP *:22 (LISTEN)
apache2 901 root 4u IPv4 TCP *:80 (LISTEN)
apache-worker 1244 root 3u IPv4 TCP 10.50.2.24:49882->185.220.101.47:8443 (ESTABLISHED)
sshd 1401 root 3u IPv4 TCP 10.50.2.24:22->10.50.1.31:50110 (ESTABLISHED)

$ cat /etc/cron.d/*
# Apache log maintenance
SHELL=/bin/sh
PATH=/usr/local/sbin:/usr/local/bin:/sbin:/bin:/usr/sbin:/usr/bin
*/5 * * * * root /usr/local/bin/apache-worker >/dev/null 2>&1

```

2007 42 Tr Raw Bytes

Output

```

10.0.0.0
10.50.1.31
10.50.2.24
185.220.101.47

```

Nothing you paste into CyberChef leaves your computer

CyberChef runs entirely as JavaScript inside your own browser. The project documentation states plainly that no recipe configuration and no input, whether text or files, is ever sent to the CyberChef web server.

That property is why it is one of the few public web tools a bank will let an analyst paste evidence into. Every other online decoder and hash calculator uploads what you give it. Know the difference, and check before you paste anything real into a website.

8.5 Step by step

1. Record exactly what triggered the alert: the process, the connection, and the behaviour that was flagged. Quote the alert.
2. Working from the five captures, write down the host's state at the time of capture: what was running, what it was connected to, and who was logged in.
3. List every indicator of compromise visible so far - process names, IP addresses, ports, file paths, account names. Use CyberChef for the addresses.
4. Find the one login in Capture 3 that should not be there, and say what makes it stand out. Compare its source address against everything else in the captures.
5. Document your evidence-handling steps in the order they were performed, and explain why volatile data was captured before anything else. Reference the volatility table in Section 8.2.

8.6 Record: initial indicators

Type	Value	Where you saw it	Why it is suspicious
Process	apache-worker	Capture 1	A root-owned process named “apache-worker” is running. This process name does not appear in the Meridian Trust standard Apache build manifest

Type	Value	Where you saw it	Why it is suspicious
IP address	185.220.101.47	Capture 2, 3, 4	It is a non-allowlisted destination and is the Source IP for an external root login and destination IP for outbound TCP connection
Port	8443	Capture 2	It is the destination port for the repeated, unauthorized outbound TCP connections
File path	usr/local/bin/apache-worker	Capture 1,5	File path and location of unapproved “apache-worker” process being executed
Account	root	Capture 1,2,3,4,5	Root account used for an unauthorized external login and is executing the rogue processes
Scheduled task	* /5 * * * * root /usr/local/bin/apache-worker >/dev/null 2>&1	Capture 5	Automatically executes the rogue script every 5 minutes and hides the output

9. Day 3, Part 2: Evidence Integrity

The disk of MTB-WEB01 was imaged at 10:31 UTC, after the volatile captures were complete. In a physical lab you would run an imaging tool and produce a file. What matters far more than the tool is what you do immediately afterwards: prove the image has not changed since you took it.

9.1 Why hashing is the whole ballgame

A cryptographic hash turns any amount of data into a fixed-length fingerprint. Two properties make it useful as evidence:

- The same input always produces the same hash. Anyone can repeat your calculation and get your answer.
- Changing the input by even one character produces a completely different hash, with no resemblance to the original.

Together those mean a hash recorded at acquisition proves the evidence was not altered afterwards. Without it, a defence lawyer, a regulator or an internal auditor can ask whether you modified the evidence, and you have no answer.

A correct conclusion built on unverified evidence is worth nothing

This is the part of forensics that new analysts skip because it feels like paperwork. It is not paperwork. It is the difference between a finding that stands up and a finding that gets thrown out.

Your grade on this lab reflects that. Your evidence handling and hash verification carry as much weight as your conclusions.

9.2 The acquisition record

```
CHAIN OF CUSTODY - CASE MTB-IR-2026-0902

Case           : MTB-IR-2026-0902
Affected asset  : MTB-WEB01 (10.50.2.24), DMZ customer portal
Alert received  : 2026-09-02 09:47 UTC
Host isolated   : 2026-09-02 10:05 UTC (network ACL, host left powered on)
Volatile capture: 2026-09-02 10:12 - 10:14 UTC
Disk acquired   : 2026-09-02 10:31 UTC
Acquired by     : Meridian Trust Bank IR Team
Media          : /dev/sda, 1 GB, MBR, single ext4 partition
Released to     : Bootcamp forensic analyst (you)

ARTEFACT HASHES RECORDED AT ACQUISITION (SHA-256)

/var/www/html/uploads/style.php
2dfc22e2fb93187b7537d9fffec52ecc94fd6416c84b78e68ff7a025e975c20c

/usr/local/bin/apache-worker
1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b

/etc/cron.d/apache-logrotate
d9bc0ee387921dcaedfc87de4a72ec42e45cf268adb2777d5b014375417cb9cc

/etc/passwd
0243860670744531f70b46cfb844a66eb8ca0de0e4b7b15dff991b3960577295
```

Those four hashes are part of your evidence. They go in your IOC list in Section 11, because a hash is how another team detects the same artefact on a different machine.

9.3 Verify a hash yourself in CyberChef

You will now perform the acquisition-and-verification cycle on a small evidence block, so you have done it rather than only read about it.

1. Open gchq.github.io/CyberChef. Clear the Recipe panel if anything is left from Section 8.
2. Search Operations for **SHA2** and drag **SHA2** into the Recipe. It is in the **Hashing** category.
3. In the SHA2 operation, make sure **Size** is set to **256**. That is the default.
4. Type this exactly into the Input panel, on one line, with no space or line break before or after it:

```
MTB-IR-2026-0902-EVIDENCE-BLOCK-01
```

5. Compare your Output against the recorded value below. They must match character for character.

```
62819c23c68b1129f103ed156fdbc8561d885d8e3a1246f439b941e73675061f
```

The screenshot shows the CyberChef interface. On the left, the 'Recipe' panel contains a 'SHA2' operation with 'Size' set to '256' and 'Rounds' set to '64'. On the right, the 'Input' panel contains the text 'MTB-IR-2026-0902-EVIDENCE-BLOCK-01'. At the bottom, the 'Output' panel displays the resulting hash: '62819c23c68b1129f103ed156fdbc8561d885d8e3a1246f439b941e73675061f'.

Hashes Match

If your hash does not match

You have a stray space, a line break, or an autocorrected character. This is the single most common problem in this section and it is easy to fix.

Click into the Input panel, press Ctrl+A then Delete to clear it completely, and type the line by hand rather than pasting it. Copying from a PDF often brings invisible characters with it.

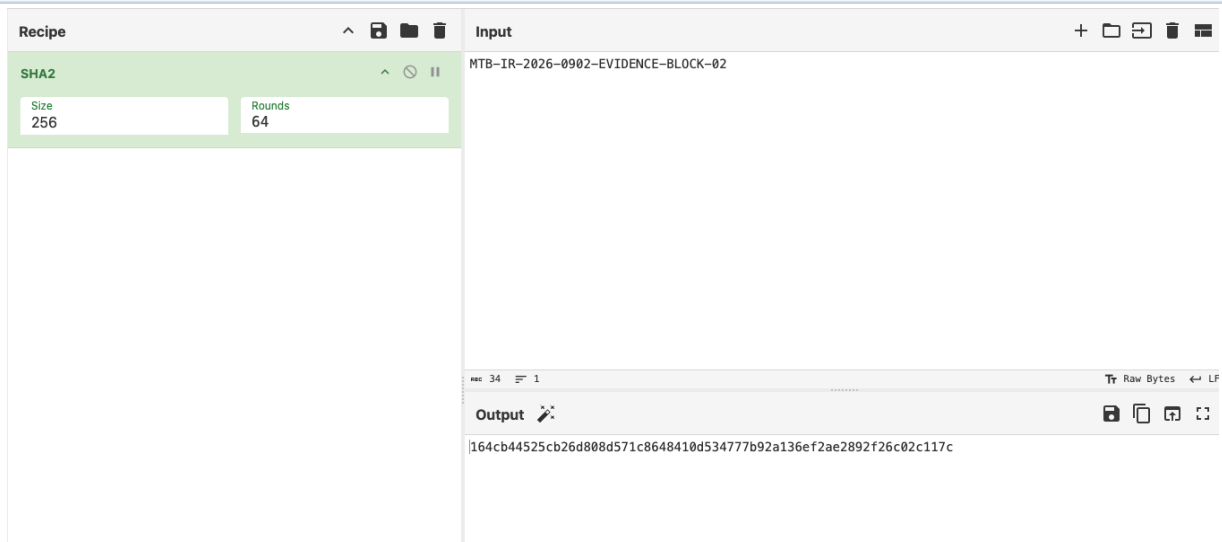
Check the character count shown at the bottom of the Input panel. It should read 34.

This is not a distraction from the lab. Evidence integrity failing because of an invisible character is a real thing that happens to real analysts, and recognising it is part of the skill.

9.4 Prove that tampering is detectable

1. Change the Input to end in `02` instead of `01`, so it reads `MTB-IR-2026-0902-EVIDENCE-BLOCK-02`.
2. Look at the new Output. Confirm it matches the value below.

164cb44525cb26d808d571c8648410d534777b92a136ef2ae2892f26c02c117c



Hashes Match

3. Compare the two hashes side by side. You changed one character out of 34. Write down how many characters of the hash stayed the same.

Input ending in 01: 62819c23c68b1129f103ed156fbdc8561d885d8e3a1246f439b941e73675061f

Input ending in 02: 164cb44525cb26d808d571c8648410d534777b92a136ef2ae2892f26c02c117c

No characters of the original hash sequence stayed the same

4. Write two sentences explaining what this demonstrates about why a hash recorded at acquisition is sufficient to prove evidence has not been altered.

This shows that changing even one character in the input causes a total and complete change of the hash output. This translates to the idea of a hash recorded at acquisition being sufficient proof of integrity of evidence. If that evidence was altered in any way, the output hash would be completely different proving that data was altered.

9.5 Doing the same thing without a browser

Every analyst should be able to hash a file without any tool beyond the operating system. These commands are built in and require nothing to be installed.

Platform	Command
Windows (Command Prompt)	certutil -hashfile evidence.txt SHA256
Windows (PowerShell)	Get-FileHash evidence.txt -Algorithm SHA256
macOS (Terminal)	shasum -a 256 evidence.txt
Linux (Terminal)	sha256sum evidence.txt

If CyberChef is unavailable for any reason, save the evidence block from Section 9.3 into a plain text file with no trailing newline and use one of these commands instead. Note in your report which method you used.

10. Day 3, Part 3: File System Timeline

Once the image was verified, the file system metadata was extracted into a timeline. Every file on a Linux file system carries four timestamps, and reading them in order reconstructs what happened on the host without needing anyone to have written a log.

10.1 Reading a forensic timeline

Letter	Timestamp	What updates it
m	Modified	The file's contents were changed
a	Accessed	The file was read
c	Changed	The file's metadata changed - permissions, ownership, or the file was created or renamed
b	Born	The file was created (also called crtime)

In the timeline below, the Type column shows which of the four timestamps fired at that moment. An entry reading machb means all four fired at once, which almost always means the file was newly created at that instant. An entry reading ma.. means the file already existed and was written to.

machb is the signature of something new appearing

When you scan the timeline, the machb entries are where to start. Each one is a file that did not exist a moment earlier. A file appearing on a production web server at half past two in the morning, on a host nobody was scheduled to be working on, is worth explaining.

10.2 The baseline: what normal looks like

This is the file system activity from the build of the host in May through to the day before the incident. It gives you something to compare against.

```
Date,Size,Type,Mode,UID,GID,File Name
Tue May 19 2026 03:10:14,10,ma..,rw-r--r--,0,0,"/etc/hostname"
Tue May 19 2026 03:10:14,116,ma..,rw-r--r--,0,0,"/etc/os-release"
Tue May 19 2026 03:10:20,220,ma..,rw-r--r--,0,0,"/etc/crontab"
```

```
Tue May 19 2026 03:12:44,226,macb,rw-r--r--,0,0,"/etc/apache2/apache2.conf"
Tue May 19 2026 03:12:58,190,macb,rw-r--r--,0,0,"/etc/ssh/sshd_config"
Tue May 19 2026 03:13:02,217,ma..,rw-r--r--,0,0,"/var/log/dpkg.log"
Tue May 19 2026 03:14:02,437,ma..,rw-r--r--,0,0,"/etc/apache2/sites-enabled/000-default.conf"
Tue May 19 2026 03:20:00,93,ma..,rw-r--r--,0,0,"/home/mtbadmin/.profile"
Tue Jun 02 2026 09:15:00,215,ma..,rw-r--r--,0,0,"/opt/mtb/asset-record.txt"
Tue Jun 02 2026 11:20:31,165,macb,rw-r--r--,0,0,"/var/www/html/index.html"
Tue Sep 01 2026 16:40:12,227,ma..,rw-r--r--,0,0,"/var/www/html/data/customer_export_q3.csv"
Tue Sep 01 2026 23:00:04,345,ma..,rw-r--r--,0,0,"/opt/mtb/installed-packages.txt"
```

10.3 The incident window: 2 September 2026

```
Date,Size,Type,Mode,UID,GID,File Name
Wed Sep 02 2026 02:15:22,236,macb,rw-r--r--,0,0,"/var/www/html/uploads/style.php"
Wed Sep 02 2026 02:21:05,571,ma..,rw-r--r--,0,0,"/var/log/apache2/error.log"
Wed Sep 02 2026 02:23:11,4096,ma..,drwxr-xr-x,0,0,"/home/svc-backup"
Wed Sep 02 2026 02:23:11,529,macb,rw-r--r--,0,0,"/etc/passwd"
Wed Sep 02 2026 02:23:11,89,macb,rw-r--r--,0,0,"/etc/group"
Wed Sep 02 2026 02:23:14,308,macb,rw-r--r--,0,0,"/etc/shadow"
Wed Sep 02 2026 02:24:48,194,macb,rw-----,0,0,"/root/.ssh/authorized_keys"
Wed Sep 02 2026 02:24:50,194,ma..,rw-----,0,0,"/home/svc-backup/.ssh/authorized_keys"
Wed Sep 02 2026 02:25:57,296,macb,rwxr-xr-x,0,0,"/usr/local/bin/apache-worker"
Wed Sep 02 2026 02:26:30,167,macb,rw-r--r--,0,0,"/etc/cron.d/apache-logrotate"
Wed Sep 02 2026 02:34:55,4096,ma..,drwxr-xr-x,0,0,"/tmp/.ICE-unix"
Wed Sep 02 2026 02:41:19,465,macb,rw-r--r--,0,0,"/root/.bash_history"
Wed Sep 02 2026 02:41:22,0,macb,rw-r--r--,0,0,"/home/mtbadmin/.bash_history"
Wed Sep 02 2026 09:02:55,2411,ma..,rw-r--r--,0,0,"/var/log/apache2/access.log"
Wed Sep 02 2026 09:45:01,732,ma..,rw-r--r--,0,0,"/var/log/syslog"
Wed Sep 02 2026 09:47:33,1809,ma..,rw-r--r--,0,0,"/var/log/auth.log"
```

10.4 Step by step

1. Copy the incident window timeline into a new spreadsheet tab called **Timeline**. Split it on commas so each field lands in its own column.
2. Add a column called **Interpretation**. Write one plain-English sentence for every row explaining what that entry means. Not "passwd was modified" - say what modifying it at that moment achieves for an attacker.
3. Mark every row whose Type is **macb**. These are files created during the incident. Group them: which are payload, which are persistence, which are access?

All MACB Files:

Payload:

/var/www/html/uploads/style.php (The initial web shell)

/usr/local/bin/apache-worker (The beaconing script)

Persistence:

/etc/cron.d/apache-logrotate (The scheduled task that keeps the beacon running)

Access:

/etc/passwd

/etc/group

/etc/shadow (Files modified to create a rogue user account and grant it high privileges)

/root/.ssh/authorized_keys (File created to allow passwordless SSH access)

/root/.bash_history (Tampered history file)

/home/mtbadmin/.bash_history (Cleared history file)

*bash history files don't give the attacker access directly, but are altered and used to hide their commands and unauthorized access

- Two files were created 27 seconds apart at 02:25:57 and 02:26:30. Explain what those two together accomplish that neither achieves alone.

The file created at 2:25:57 is “apache-worker” and is the malicious payload, the file created at 2:26:30 is “apache-logrotate” and is the scheduled cron job telling the system to run it. They operate together as a payload sitting on the disk does nothing if it isn't executed, and if it is run manually, it will die if the system reboots. On the other hand, a cron job is useless without a script to run. Together, they ensure the attacker maintains a constant connection to the host.

- Look at the two entries at 02:41:19 and 02:41:22. One file has content, the other is zero bytes. Explain what the attacker was doing and why one attempt worked while the other did not.

The attacker was attempting to wipe the logs of the bash history files of root and mtadmin by changing it to zero bytes. The attempt to clear mtbadmin's history at 02:41:22 worked, leaving it at zero bytes. However, for root, as the attacker was actively operating inside a root shell, when he exited the bash session, the shell automatically pushed the history held in active memory back to the disk. The attacker cleared the file on disk, but when they logged out, bash repopulated /root/.bash_history with 465 bytes of the commands they executed exposing their root activity.

- Compare the incident window against the baseline in Section 10.2. Write two sentences on what makes the incident activity obviously abnormal for this host.

While normal baseline activity involves regular hours and scheduled automated services, this timeline shows repeated privileged system configurations from 2:21:05 to 2:26:30 AM. Also, standard system administration does not involve creating the unrecognized file “apache-worker” in /usr/local/bin and altering core identity management files (/etc/passwd, /etc/shadow).

The size column is evidence too

One entry in the incident window has a size of 0 bytes. Empty files are not normally interesting - this one is, because of which file it is and because the equivalent file for the other account is not empty.

When you find it, ask what would have to happen for that file to end up empty, and whether the same thing happened to the equivalent file for the other account.

11. Day 3, Part 4: Log Correlation and Indicator Analysis

The timeline tells you what changed and when. The logs tell you who did it and how. Correlating the two is what turns a list of suspicious files into an account of an intrusion.

11.1 Apache access log

```
10.50.1.31 - - [01/Sep/2026:14:02:11] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
10.50.1.44 - - [01/Sep/2026:14:07:53] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
10.50.1.31 - - [01/Sep/2026:16:41:02] "GET /data/customer_export_q3.csv" 200 341 "Mozilla/5.0"
185.220.101.47 - - [02/Sep/2026:02:08:14] "GET / HTTP/1.1" 200 218 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:08:19] "GET /server-status" 404 452 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:08:24] "GET /.git/config" 404 452 "Nmap Scripting Engine"
185.220.101.47 - - [02/Sep/2026:02:09:02] "GET /icons/README" 200 1401 "curl/7.81.0"
```

```
185.220.101.47 - - [02/Sep/2026:02:12:33] "GET /cgi-bin/.%2e/.%2e/.%2e/etc/passwd" 200 622 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:12:41] "GET /cgi-bin/.%2e/.%2e/.%2e/etc/apache2/apache2.conf" 200 289 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:13:01] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 89 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:13:48] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 142 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:15:22] "POST /cgi-bin/.%2e/.%2e/.%2e/bin/sh" 200 61 "curl/7.81.0"
185.220.101.47 - - [02/Sep/2026:02:17:40] "POST /uploads/style.php" 200 34 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:19:12] "POST /uploads/style.php" 200 812 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:21:05] "POST /uploads/style.php" 200 57 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:26:30] "POST /uploads/style.php" 200 19 "Mozilla/5.0 (X11; Linux x86_64)"
185.220.101.47 - - [02/Sep/2026:02:34:55] "POST /uploads/style.php" 200 128 "Mozilla/5.0 (X11; Linux x86_64)"
10.50.1.31 - - [02/Sep/2026:08:14:07] "GET / HTTP/1.1" 200 218 "Mozilla/5.0 (Windows NT 10.0)"
```

Read the status codes, not just the URLs

Two requests at 02:12 use the same technique. Look at what came back: 200 with a body of 622 bytes, then 200 with 289 bytes.

A 200 means the server returned content. Compare those against the 404s two minutes earlier, and against the size of the real `/etc/passwd` file in the timeline. The scanner findings from Day 1 tell you what those requests were exploiting.

11.2 Authentication log

```
Sep  1 09:14:22 mtb-web01 sshd[1102]: Accepted publickey for mtbadmin from 10.50.1.31 port 49221
Sep  2 02:20:47 mtb-web01 pkexec[1207]: pam_unix(polkit-1:session): session opened for user root(uid=0) by (uid=33)
Sep  2 02:21:05 mtb-web01 pkexec[1207]: www-data: Executing command [USER=root] [TTY=unknown]
[CWD=/var/www/html/uploads] [COMMAND=/bin/sh]
Sep  2 02:21:05 mtb-web01 polkitd[812]: Registered Authentication Agent for unix-process:1207
Sep  2 02:23:11 mtb-web01 useradd[1219]: new user: name=svc-backup, UID=0, GID=0,
home=/home/svc-backup, shell=/bin/bash
Sep  2 02:23:11 mtb-web01 useradd[1219]: WARNING: UID 0 is already in use by user root
Sep  2 02:23:14 mtb-web01 chpasswd[1221]: password changed for svc-backup
Sep  2 02:24:48 mtb-web01 sshd[1230]: Accepted publickey for root from 185.220.101.47 port 44118
Sep  2 02:30:01 mtb-web01 CRON[1244]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 02:35:01 mtb-web01 CRON[1251]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 02:40:01 mtb-web01 CRON[1259]: pam_unix(cron:session): session opened for user root(uid=0)
Sep  2 09:47:33 mtb-web01 sshd[1401]: Accepted publickey for mtbadmin from 10.50.1.31 port 50110
```

Three lines in this log are the whole privilege story

`uid=33` is the `www-data` account - the unprivileged identity a web server runs as. Something running as `uid 33` opened a root session at 02:20:47 through `pkexec`.

Look at what your Day 1 assessment said was installed on this host, and which finding matched it. The privilege escalation was not a mystery; it was on your list.

The `useradd` warning two minutes later is the system telling you plainly that something is wrong. A new account was given the same UID as root. That is not a configuration mistake, it is a backdoor.

11.3 System log

```
Sep  2 02:23:11 mtb-web01 systemd: Created slice User Slice of UID 0.
Sep  2 02:26:31 mtb-web01 cron[812]: (*system*) RELOAD (/etc/cron.d/apache-logrotate)
Sep  2 02:30:01 mtb-web01 CRON[1244]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 02:35:01 mtb-web01 CRON[1251]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 02:40:01 mtb-web01 CRON[1259]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 08:00:01 mtb-web01 CRON[1310]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
Sep  2 09:45:01 mtb-web01 CRON[1396]: (root) CMD (/usr/local/bin/apache-worker >/dev/null 2>&1)
```

11.4 Recovered files

Two files recovered from the image. The first is the suspicious file created at 02:15:22. The second is root's shell history, which the attacker did not clear.

/var/www/html/uploads/style.php

```
<?php
/* LAB ARTIFACT - INERT - Meridian Trust Bank forensics exercise */
$k = "a4f1";
$p = "TVRCLUXBQi1BUlRJRkFDVDogc2ltdWxhdGVkIHd1YnNoZWxsIHBheWxvYWQsIG5vIGV4ZWN1dGFibGUgY29udGVudA==";
/* payload handler removed for lab safety */
?>
```

/root/.bash_history

```
id
uname -a
cat /etc/os-release
which curl wget python3
useradd -o -u 0 -g 0 -m -s /bin/bash svc-backup
echo 'svc-backup:REDACTED' | chpasswd
mkdir -p /root/.ssh
cat >> /root/.ssh/authorized_keys
chmod 600 /root/.ssh/authorized_keys
cat > /usr/local/bin/apache-worker
chmod +x /usr/local/bin/apache-worker
cat > /etc/cron.d/apache-logrotate
systemctl restart cron
mkdir -p /tmp/.ICE-unix
tar czf /tmp/.ICE-unix/x /var/www/html/data
ls -la /tmp/.ICE-unix
history -c
```

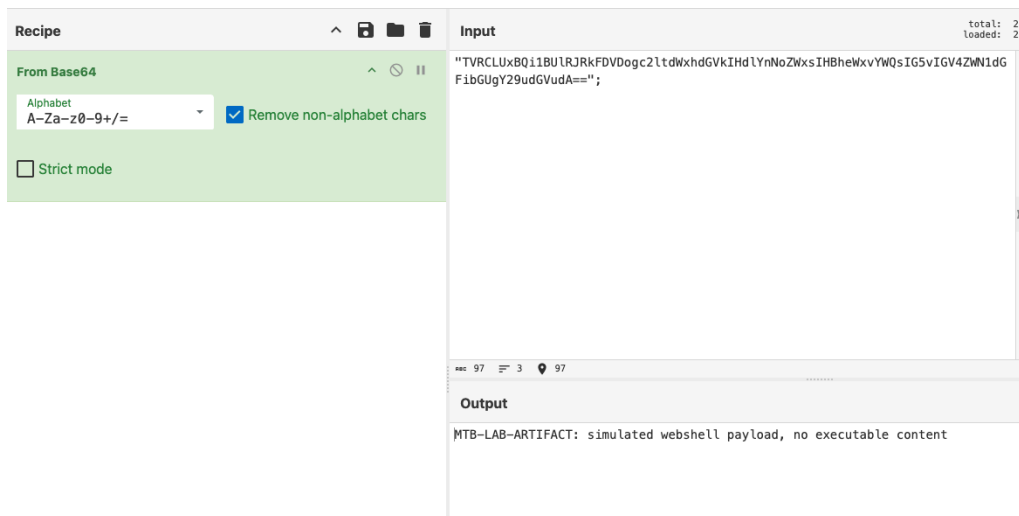
/root/.ssh/authorized_keys

```
ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAAIILABPLACEHOLDERKEYNOTREALDONOTUSE00000000 mtbadmin@mtb-jump01
ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AAAAIILABPLACEHOLDERKEYNOTREALATTACKER00000000 attacker@blackfin
```

11.5 Decoding the suspicious file

The PHP file contains a base64-encoded string. Encoding is not encryption - it is a way of representing data as plain text, and it is trivially reversible. Attackers use it constantly to make payloads harder to spot in a casual read.

1. In CyberChef, clear the Recipe panel.
2. Search Operations for **From Base64** and drag **From Base64** into the Recipe. It is in the **Data format** category.
3. Paste the encoded string into the Input panel. It is printed across two lines in Section 11.4 for page width - join it into one line with no space where it wraps.
4. Read the Output. Record what it decodes to.



MTB-LAB-ARTIFACT: simulated webshell payload, no executable content

- Write two sentences on why an attacker would encode a payload this way, and why encoding provides no actual security.

An attacker would encode a payload this way to make it harder for security teams or automated tools to recognize malicious commands during a casual inspection. However, encoding provides no actual security because it is just a different way of representing data as plain text rather than actually encrypting it, allowing anyone to trivially reverse the process.

What this file is, and what it is not

In a real incident this file would be a webshell: a small script uploaded to a web server that lets an attacker run commands through ordinary web requests. Look at the access log entries that POST to it after 02:17:40 and you can see it being used.

The file in this lab has had its working code removed and replaced with a marker string. You are learning to recognise the shape of a webshell and the traces it leaves, not handling live malicious code.

11.6 Building the consolidated IOC list

An IOC list is the deliverable another team acts on. Its whole purpose is to let someone who was not part of your investigation search their own systems for the same activity. That means it must be precise, complete, and safe to circulate.

- Collect every indicator from the timeline, the logs, the recovered files and the acquisition record. Use CyberChef's **Extract IP addresses** on the log sections to make sure you missed nothing.
- Sort them into types: network, file, hash, account, persistence.
- For each indicator, record where you found it. An indicator without a source is not evidence.
- Defang the network indicators before circulating the list. In CyberChef, search for **Defang IP** and drag **Defang IP Addresses** into the Recipe. Paste your address list in.

Why defanging matters

Defanging rewrites 185.220.101.47 as 185[.]220[.]101[.]47 so that no mail client, chat app, ticketing system or document viewer turns it into a clickable link.

IOC lists get emailed, pasted into tickets and forwarded widely. Somebody eventually clicks. Defanging is a one-second habit that prevents a colleague from connecting to attacker infrastructure by accident.

11.7 Record: consolidated IOC list

Type	Indicator (defanged)	Source	What it detects
Network	185[.]220[.]101[.]47	Capture 2 (ss -tunap), Capture 3 (last), Capture 4 (lsnf), Apache Access log	Attacker established command and control connection and unauthorized external login
Network	TCP/8443	Capture 2 (ss -tunap), Capture 4 (lsnf)	Outbound destination port used by the malicious beacon
Network	Nmap Scripting Engine	Apache Access Log	String indicating the use of automated scanning tools
Network	curl/7.81.0	Apache Access Log	String utilized during the directory traversal and remote code execution
File path	/usr/local/bin/apache-worker	Capture 1 (ps -ef), Capture 5 (cat /etc/cron.d/*), MACB timeline	Malicious executable file acting as a reverse shell/beacon
File path	/var/www/html/uploads/style.php	File System Timeline	Initial web shell dropped by the attacker to execute remote commands
File hash	/usr/local/bin/apache-worker: 1ebcce99569f4337f9ee3746b7b095a9f54abfe3036ef06014b1a00ea4d15d4b	Acquisition Record	Shows identical copies of the beacon malware across system endpoints
File hash	/var/www/html/uploads/style.php 2dfc22e2fb93187b7537d9ffec52ecc94fd6416c84b78e68ff7a025e975c20c	Acquisition Record	Shows identical web shells uploaded to other web servers within the system
Account	root	Capture 3 (last)	External use of the root account from attacker IP
Account	svc-backup	File System Timeline, Authentication Log, Recovered Files (/root/.bash_history)	Rogue user account created by the attacker and assigned UID 0 to grant it root privileges

Type	Indicator (defanged)	Source	What it detects
Persistence	/etc/cron.d/apache-logrotate	System Log, Recovered Files (/root/.bash_history)	Scheduled cron task created by the attacker to consistently execute the apache-worker payload
Persistence	ssh-ed25519 AAAAC3NzaC11ZDI1NTE5AA AAAILABPLACEHOLDERKE YNOTREALATTACKER00000 00 attacker@blackfin	Recovered Files (/root/.ssh/authorized_keys)	Attacker's public SSH key placed in /root/.ssh/authorized_keys to establish passwordless root access
Other	/tmp/.ICE-unix	File System Timeline, Recovered Files (/root/.bash_history)	Hidden staging directory utilized by the attacker to exfiltrate the /var/www/html/data directory
Other	TVRCLUxBQi1BUIRJrkFDVD ogc2ltdWxhdGVkIHdlYnNoZW xsIHBheWxvYWQsIG5vIGV4Z WN1dGFibGUgY29udGVudA= =	Recovered Files (/var/www/html/uploads /style.php)	Base64-encoded string found inside the web shell payload

11.8 Step by step

1. Correlate the host's local logs against the monitoring alert timeline in Section 8.1. Confirm the sequence of events lines up, and note anything that does not.

The SOC alert noted an outbound connection starting at approximately 02:31 UTC. The system logs (/var/log/syslog) confirm this, showing the cron service executing the /usr/local/bin/apache-worker script for the first time at 02:30:01, and then exactly every 5 minutes after that. The authentication logs confirm the incident response team logging in (mtbadmin at 09:47:33) exactly when the alert fired, which led to the isolation at 10:05 UTC. There are no notable discrepancies.

2. Trace the suspicious process and network activity from Section 8.3 back through the logs. What was it connecting to, when did it start, and what caused it to start?

The process apache-worker maintained a TCP connection to the attacker's IP address: 185.220.101.47 on port 8443. The process first executed at 02:30:01 UTC on September 2, 2026 and was triggered by a malicious cron job file (/etc/cron.d/apache-logrotate) created by the attacker at 02:26:30 and executed every 5 minutes.

3. Establish how the attacker moved from the web server's unprivileged account to root. Name the mechanism and cite the exact log lines.

The attacker moved from the unprivileged www-data account (UID 33) to root (UID 0) by exploiting the Polkit pkexec vulnerability(CVE-2021-4034), identified as a finding on this host during the initial vulnerability scan.

Exact Log Lines:

Sep 2 02:20:47 mtb-web01 pkexec[1207]: pam_unix(polkit-1:session): session opened for user root(uid=0) by (uid=33)

Sep 2 02:21:05 mtb-web01 pkexec[1207]: www-data: Executing command [USER=root] [TTY=unknown] [CWD=/var/www/html/uploads] [COMMAND=/bin/sh]

4. Identify every persistence mechanism the attacker installed. There are three distinct ones. Each would survive something different - a reboot, a password reset, a process being killed. Say which survives what.

Cron Job (/etc/cron.d/apache-logrotate): Survives a process being killed. If the system restarts or the apache-worker process is killed, the cron job will automatically restart the payload at the next 5-minute interval.

SSH Key (/root/.ssh/authorized_keys): Survives a password reset. If the root password is changed to lock the attacker out, the attacker can still log in directly through SSH keys bypassing password authentication.

Rogue Account (svc-backup with UID 0): Survives a reboot. While active reverse shells or processes in memory are cleared when a system restarts, user identities and passwords are saved to the disk allowing the attacker to use this backdoor account to log back in as soon as the system powers on.

5. Compile your consolidated IOC list into the table in Section 11.7.

12. Day 3, Part 5: Root Cause and Forensic Report

You now know what happened. The remaining question is the one the bank actually cares about: why was this possible, and what stops it happening again.

12.1 Finding the root cause

Root cause is not the webshell, and it is not the beacon. Those are consequences. Root cause is the specific weakness that let an unauthenticated attacker on the internet get their first foothold on a bank's DMZ web server.

1. Look at the first request in the access log that returned a 200 with a body, from the external address, using an unusual URL structure. Note the time.

185.220.101.47 - - [02/Sep/2026:02:12:33] "GET /cgi-bin/.%2e/.%2e/.%2e/.%2e/etc/passwd" 200 622 "curl/7.81.0"

2. Match that request pattern against the findings from your Day 1 assessment. Which finding describes exactly this technique against exactly this software version?

This is a path traversal attack utilizing URL encoding (.%2e) to access system files which match the findings for Day 1 with Apache HTTP Server Path Traversal and RCE targeting Apache httpd version 2.4.49.

3. Record the finding ID and the CVE. Check your Day 2 work: where did that finding rank after you applied the KEV cross-reference, and where would it have ranked on CVSS alone?

Finding: MTB-0002 , CVE-2021-41773 (CVSS 7.5).

This vulnerability gave the attacker their initial foothold. In our Day 2 threat-intelligence prioritization, this finding moved up to Rank #5 because it was listed in the KEV catalog as actively exploited. If we had prioritized based on CVSS scores alone, this CVSS 7.5 vulnerability would have moved far down the list, overshadowed by non-exploitable vulnerabilities such as with a CVSS 9.8

- Identify the second vulnerability in the chain - the one that took the attacker from the web server account to root. It is also on your Day 1 list.

The second vulnerability in the chain, which allowed the attacker to elevate their privileges from the unprivileged www-data web server account to full root access, was MTB-0003: polkit pkexec Local Privilege Escalation (PwnKit) (CVE-2021-4034).

- Note the configuration on MTB-WEB01 that made the first vulnerability exploitable. It is printed in the evidence pack in Section 5.3. The software version alone was not sufficient.

The software version alone (Apache 2.4.49) was not sufficient to allow this compromise. The root cause configuration that made the path traversal exploitable was found in the Apache 000-default.conf file. By explicitly setting Require all granted on the root directory, the server overrode Apache's default secure access controls allowing path traversal.

```
$ curl -sI http://10.50.2.24/
HTTP/1.1 200 OK
Server: Apache/2.4.49 (Ubuntu)

$ dpkg -l | grep -E 'apache2 |policykit'
ii  apache2      2.4.49-1ubuntu1      amd64  Apache HTTP Server
ii  policykit-1  0.105-31             amd64  authorization framework

$ grep -A2 'ScriptAlias' /etc/apache2/sites-enabled/000-default.conf
ScriptAlias /cgi-bin/ /usr/lib/cgi-bin/
<Directory />
    Require all granted
```

This is why Days 1 and 2 mattered

Every part of this compromise was on your assessment list six days before it happened. Nothing here was a zero-day and nothing was unknowable.

The vulnerability that gave the attacker their foothold was the one that moved up your priority list when you applied threat intelligence, and down it if you had ranked on CVSS alone.

Say that explicitly in your report. It is the strongest argument you will ever make for why vulnerability prioritisation is worth doing properly.

12.2 Building the incident timeline

Your report needs one merged timeline covering the whole incident, drawing on every source. Build it in your spreadsheet with these columns:

Time (UTC)	Event	Evidence source	Confidence
2:08:14	Attacker initiated automated vulnerability scanning using Nmap.	Apache access log	Inferred
2:12:33	Attacker successfully exploited path traversal to read the /etc/passwd file.	Apache access log	Inferred
2:15:22	Attacker achieved remote code execution to write the initial style.php web shell payload to the disk.	Apache access log, File timeline	Confirmed
2:17:40	Attacker began interacting with the newly	Apache access log	Inferred

Time (UTC)	Event	Evidence source	Confidence
	dropped web shell via a web browser.		
2:20:47	Attacker escalated privileges from the unprivileged www-data account to root using the PwnKit (pkexec) exploit.	Authentication log	Inferred
2:23:11	Attacker created a rogue backdoor account (svc-backup) and assigned it UID 0 to grant it root privileges.	File timeline, Authentication log, System log, .bash_history	Confirmed
2:24:48	Attacker established persistent root access by dropping an SSH key and authenticated directly via SSH.	File timeline, Authentication log, .bash_history	Confirmed
2:25:57	Attacker created the malicious apache-worker beacon/reverse shell binary on the disk.	File timeline, .bash_history	Confirmed
2:26:30	Attacker created the apache-logrotate cron job to ensure persistent automated execution of the beacon.	File System log, .bash_history	Confirmed
2:34:55	Attacker created the hidden /tmp/.ICE-unix staging directory and compressed data for exfiltration.	File timeline, .bash_history	Confirmed
2:41:19	Attacker ended their root shell session, writing their command history to the disk.	File timeline, recovered .bash_history file	Confirmed
9:47:12	Incident responder (mtbadmin) logged into the host to investigate the alerts.	Authentication log, terminal capture (last/who)	Confirmed

The Confidence column matters. Mark each entry as Confirmed if two independent sources agree, or Inferred if you are reasoning from a single source. An honest report distinguishes the two.

12.3 Prevention and monitoring recommendations

Recommendations must be specific enough that someone could implement them from your report alone. Cover all three of these:

- **Patching** - name the packages and the versions to upgrade to, and address why a host built in May was still running software with a publicly known, actively exploited flaw in September.
- **Configuration** - name the specific Apache directives that made the flaw exploitable and what they should be set to instead.
- **Detection** - at least one recommendation must be an endpoint protection or monitoring improvement that would have caught this faster. The compromise happened at 02:15 and was detected at 09:47. Explain what would have shortened that gap and by roughly how much.

Meridian Trust Bank: Incident Response Forensic Report

Case: MTB-IR-2026-0902

Executive Summary:

On September 2, 2026, an external attacker breached the Meridian Trust Bank DMZ web server (MTB-WEB01) by exploiting a known vulnerability to gain unauthorized access. The attacker successfully escalated their privileges to assume full administrative control, allowing them to install persistent backdoors and stage data for exfiltration, putting sensitive bank operations and data at high risk. The incident response team successfully detected the attacker's automated network beacon, isolated the compromised host from the broader network, and preserved critical forensic evidence. The immediate threat has been contained, and remediation steps are underway to close the exploited vulnerabilities and restore secure operations.

Evidence Handling:

Evidence was captured complying to the order of volatility to ensure system state data was not lost. Volatile data held in RAM, including running processes (ps -ef), active network connections (ss -tunap), logged-in users (who), and open network files (lsof) was captured first. Only after this live state was secured did responders capture non-volatile physical disk evidence, such as scheduled tasks (/etc/cron.d/).

To ensure evidence integrity, SHA-256 hashes were calculated at the start of the acquisition record.

Hash of Evidence Block 01 (MTB-IR-2026-0902-EVIDENCE-BLOCK-01):
62819c23c68b1129f103ed156fbd8561d885d8e3a1246f439b941e73675061f

Incident Timeline:

See Incident Timeline Above

Root Cause:

The attack relied on a chain of two specific vulnerabilities, both of which were identified during the Day 1 security assessment.

Initial Foothold: The attacker bypassed web directories using an Apache HTTP Server Path Traversal flaw (CVE-2021-41773, Finding ID: MTB-0002). This flaw was made exploitable because the host's Apache 000-default.conf configuration insecurely applied a Require all granted directive to the root directory.

Privilege Escalation: Once executing commands as the unprivileged www-data web server account, the attacker escalated to root using the polkit pkexec Local Privilege Escalation (PwnKit) vulnerability (CVE-2021-4034, Finding ID: MTB-0003).

The vulnerability that gave the attacker their foothold (MTB-0002) was the exact vulnerability that moved up our priority list when threat intelligence (KEV) was applied in Day 1 findings, and would have been incorrectly deprioritized if ranked on static CVSS scores alone.

Indicators of Compromise:

See Section 11.6 for Consolidated IoC List

Recommendations

Patching

Apache HTTP Server: Upgrade the apache2 package to version 2.4.51 or later to remediate the path traversal and RCE vulnerabilities (CVE-2021-41773,CVE-2021-42013).

Polkit: Upgrade the policykit-1 package to the latest distribution-patched version to remediate the PwnKit privilege escalation vulnerability (CVE-2021-4034).

Lifecycle Management: This host was built in May but was running actively exploited software in September. Implement an automated vulnerability management lifecycle to ensure production servers receive routine patch deployments rather than relying on static deployment images.

Configuration

Apache: Immediately revert the configuration in 000-default.conf from <Directory/> Require all granted to the secure default of <Directory/> Require all denied.

System Hardening: Apply mandatory access control profiles (e.g., AppArmor enforcement) to the www-data service. This restricts the web server process from executing interactive shells (/bin/sh) or writing payloads outside of strictly required directories, mitigating the impact even if a web vulnerability is exploited.

Detection

The attacker was unnoticed in the network for 7.5 hours (from 02:15 to 09:47) before a periodic beacon triggered an alert. The following detections would have alerted the SOC at the exact moment of creation:

Rogue Account Creation (UID 0): Forward authentication logs to SIEM. Configure an immediate, high-severity alert for any useradd event resulting in a UID 0 assignment. This would have detected the intrusion at 02:23 (The moment the account was created)

SSH Key Tampering: Implement file integrity monitoring (FIM) to monitor /root/.ssh/authorized_keys. Generate a critical alert for any file modification that occurs outside of an approved change management window.

Cron Job Tampering: Implement FIM alerting for any file creation or modification within the /etc/cron.d/ directory to catch automated persistence instantly.

Detection is where most of the value is

The attacker was inside for seven and a half hours before anyone noticed, and they were only noticed because the beacon was noisy and predictable. A quieter one would still be running.

For each of the three persistence mechanisms you identified, ask what would have alerted on it at the moment it was created rather than hours later. Write those down as concrete detections, not as "improve monitoring".

12.4 Report structure to follow

Section	Contents
Executive summary	Three to five sentences for a non-technical stakeholder. What happened, what was at risk, what has been done.

Section	Contents
Evidence handling	How evidence was preserved, in what order, and how integrity was verified. Include your hashes.
Incident timeline	The merged timeline from Section 12.2, with confidence marked.
Root cause	The vulnerability chain, the CVEs, the configuration that enabled it, and the link to your Day 1 finding IDs.
Indicators of compromise	Your defanged IOC list from Section 11.7.
Recommendations	Patching, configuration and detection, each specific and actionable.

13. Deliverables

Submit the following through the class shared drive.

13.1 Days 1 and 2

- Your severity breakdown and top ten list.
- Your five manually validated findings, including at least one proved false positive and at least one proved true positive, with the version ranges and evidence you based each verdict on.
- Your KEV cross-reference table covering all ten CVEs.
- Your re-prioritised findings list, with a written explanation of at least one finding whose position changed.
- Your full vulnerability assessment report with remediation plan, compensating controls, false positive section, and recommended scan cadence.

13.2 Day 3

- Your evidence preservation record, including the volatile data captured, the order it was captured in, and your order-of-volatility reasoning.
- Your hash verification work from Section 9, including both hash values and what changing one character demonstrated.
- Your annotated file system timeline.
- Your log correlation notes and your consolidated, defanged IOC list.
- Your final forensic report with root cause, full merged timeline, and prevention recommendations.

14. Reflection Questions

Answer briefly in your own words, three to five sentences each. These are graded on reasoning, not length.

14.1 Days 1 and 2

1. Why can a vulnerability assessment not stop at the raw scanner output? Give a specific example from this lab of what manual validation changed.

Raw scanner output can contain false positives based on generalized signatures without understanding the host's actual configuration. In this lab, the raw scanner flagged a path traversal vulnerability based on the Apache version, but

manual validation of the 000-default.conf file confirmed the Require all granted directive made it actually exploitable. Relying solely on the scanner would miss environmental context or result in false alarms.

2. Give an example from this scan where CVSS score alone would have led you to the wrong priority order, and explain what threat intelligence corrected.

The Apache path traversal vulnerability had a CVSS score of 7.5, which would have placed it below 9.8 vulnerabilities if ranked purely by severity. However, cross-referencing with the CISA Known Exploited Vulnerabilities (KEV) catalog revealed that this specific CVE was actively being exploited in the wild. Threat intelligence elevated this real-world threat to the top of the remediation list, proving that active exploitation is a necessary prioritization metric.

3. All three false positives in this scan were in the KEV catalog. Explain why that combination is more dangerous to a security team than a false positive that is not in KEV.

False positives that appear in the KEV catalog create an idea of immediate real-world danger, triggering emergency incident response protocols. A security team will drop standard operations for a KEV-listed finding, wasting time and resources on emergency remediation for a non-existent threat. While a non-KEV false positive might just sit in a backlog, a KEV-listed false positive disrupts operations.

4. Why is a compensating control still valuable even when it does not fix the underlying vulnerability?

A compensating control breaks the attack chain by removing the attacker's ability to reach or exploit the vulnerability, even if the malicious code remains on the system. This provides immediate operational protection and gives the engineering team time to test and deploy a software patch.

14.2 Day 3

1. Why does the order you collect evidence in matter, and what specifically would have been lost if the disk had been imaged before the running processes and network connections were captured?

Evidence collection must follow the order of volatility because interacting with a system inherently alters it, and powering it down destroys data held in volatile memory. If the disk had been imaged first, the active apache-worker process and the established outbound network connection to the command and control IP would have been permanently wiped leaving no evidence to recover.

2. How did correlating the host logs with the monitoring alert change or confirm your understanding of what happened?

Correlating logs with the alert transformed allowed me to piece together the incident timeline and narrative of the attacker. The syslog confirmed a cron job triggered the 02:31 alert, but the authentication and access logs revealed the attacker's initial entry and privilege escalation happened nearly fifteen minutes earlier. This proved there was a deeper, pre-existing compromise rather than the initial point of entry.

3. Why is identifying the root cause more valuable to the bank than simply removing the malicious process and the backdoor account?

Removing the malicious process and the backdoor account only treats the symptoms of the breach, leaving the system still completely exposed. If the root cause, the insecure Apache configuration and unpatched software, remains unfixed, the attacker will simply re-exploit the server immediately after the remediation steps. Identifying and fixing the root cause is the only way to secure the environment and prevent another identical attack.

4. The attacker cleared one shell history file and missed another. What does that tell you about relying on any single evidence source, and how did the timeline let you detect the deletion even though the contents were gone?

Relying on a single evidence source creates a single point of failure. Attackers actively manipulate logs so investigations must cross-reference multiple sources to get a complete understanding of the compromise. The timeline revealed that the administrative bash history file had its modification timestamp (MACB) updated simultaneously with

the attacker's logout at 02:41, despite the file being completely empty. The difference of a recent modification on an empty file shows that there was a deletion attempt.

15. Grading Alignment

This lab counts toward your lab-completion and documentation-quality grading criteria.

16. Completion Checklist

Work down this list before you submit. Every line is something that is graded. If you cannot tick a line, that is where to spend your remaining time.

16.1 Day 1

Done	Item
	All 37 findings copied into your spreadsheet
	Severity counts recorded, and the five counts add up to 37
	Top ten by CVSS recorded, with the host for each
	At least one finding flagged as suspicious, with a reason written down
	Five findings picked for validation, covering a mix of severities
	Each of the five looked up on NVD, with the affected range recorded
	At least one false positive proved, quoting the range and the detected version
	At least one true positive proved, with the exact confirming evidence
	Any severity changes noted, with the reason for each

16.2 Day 2

Done	Item
	All ten CVEs checked against KEV, including the ones you disproved
	KEV date added recorded for every CVE that is listed
	Findings re-prioritised so confirmed exploitation outweighs CVSS alone
	False positives removed from the priority list entirely
	At least one position change explained in writing
	Publication date and KEV date recorded for your top finding, with the gap noted
	Report written with all six sections from Section 7.3
	Every finding has a specific fix, not "patch it"
	Compensating control proposed for anything that cannot be patched now
	Scan cadence recommended and justified
	Executive summary written with no CVE numbers and no CVSS scores

16.3 Day 3

Done	Item
	Alert trigger recorded and quoted
	All five volatile captures reviewed and the host state written up
	Initial IOC table filled in, all six rows
	The out-of-place login in Capture 3 identified and explained
	Order-of-volatility reasoning written, referencing the volatility table
	Both hash values from Section 9 reproduced and matched
	Tampering demonstration completed and written up
	Timeline copied into a spreadsheet and split into columns
	Every timeline row given a plain-English interpretation
	All macb rows identified and grouped as payload, persistence or access
	The two files created 27 seconds apart explained together
	The zero-byte file explained
	Privilege escalation mechanism named, with the log lines cited
	All three persistence mechanisms identified, with what each survives
	Base64 payload decoded and recorded
	Consolidated IOC list completed and defanged
	Root cause identified, with the CVE and your Day 1 finding ID
	Second vulnerability in the chain identified
	The enabling configuration named
	Merged incident timeline built, with confidence marked on every row
	Recommendations cover patching, configuration and detection
	Executive summary written for a non-technical reader

Before you submit

Read your two executive summaries out loud. If you stumble, or if you used a word a branch manager would not know, rewrite them.

Check that every factual claim about a CVE in your reports came from NVD or the CISA catalog and not from memory or from an AI assistant.

Check that no live, clickable attacker address survives anywhere in your IOC list.

Appendix A: Reference Data and Troubleshooting

Everything in this appendix is here so that nothing can stop you. If a website is slow, blocked or down, the values you would have looked up are printed below. Use them, and note in your report that you used the printed reference rather than the live source.

A.1 CVE affected version ranges

These are the affected ranges as published by the vendors and recorded on NVD. Compare each against the detected version in Section 3.2.

CVE	Affected versions	Detected on host	Verdict
CVE-2021-42013	Apache httpd 2.4.49 and 2.4.50 only	Apache 2.4.49	In range
CVE-2021-41773	Apache httpd 2.4.49 only	Apache 2.4.49	In range
CVE-2021-4034	polkit pkexec, all versions from 2009 until the January 2022 fix	policykit-1 0.105-31	In range
CVE-2020-11022	jQuery 1.2 up to but not including 3.5.0	jQuery 3.4.1	In range
CVE-2011-2523	vsftpd 2.3.4 only (backdoored release tarball)	vsftpd 2.3.4	In range
CVE-2014-6271	GNU bash through 4.3	GNU bash 5.1.16	OUT of range
CVE-2017-7494	Samba 3.5.0 up to but not including 4.6.4	Samba 4.15.13	OUT of range
CVE-2023-4911	glibc 2.34 through 2.37 in affected builds	glibc 2.35	In range
CVE-2019-0708	Windows XP, Server 2003, Vista, 7, Server 2008, Server 2008 R2	Windows Server 2019	OUT of range
CVE-2020-1472	Windows Server 2008 R2 through 2019, unpatched since Aug 2020	Windows Server 2019, last update June 2020	In range

Do the NVD lookups yourself first. This table is a safety net and a way to check your work, not a shortcut past the exercise. Your report must show the reasoning, not just the verdict.

A.2 KEV catalog status

Status as recorded in the CISA Known Exploited Vulnerabilities catalog. Verify each one live on the CISA site during Section 5.

CVE	CVSS	KEV status	Date added to KEV
CVE-2021-42013	9.8	In KEV	2021-11-03
CVE-2021-41773	7.5	In KEV	2021-11-03
CVE-2021-4034	7.8	In KEV	2022-06-27
CVE-2020-11022	6.1	Not in KEV	-
CVE-2011-2523	9.8	Not in KEV	-
CVE-2014-6271	9.8	In KEV	2022-01-28

CVE	CVSS	KEV status	Date added to KEV
CVE-2017-7494	9.8	In KEV	2023-03-30
CVE-2023-4911	6.5	In KEV	2023-11-21
CVE-2019-0708	9.8	In KEV	2021-11-03
CVE-2020-1472	8.8	In KEV	2021-11-03

A.3 CyberChef quick reference

Every operation used in this lab, where to find it, and what it does. Type the operation name into the search box at the top left of the Operations panel.

Operation	Category	Used in	What it does
Extract IP addresses	Extractors	7.4, 10.6	Pulls every IPv4 and IPv6 address out of a block of text
Sort	Utils	7.4	Sorts the output lines
SHA2	Hashing	8.3, 8.4	Calculates a SHA-256 fingerprint of the input
From Base64	Data format	10.5	Decodes a base64 string back to readable text
Defang IP Addresses	Networking	10.6	Rewrites addresses so they cannot become clickable links

Three CyberChef habits worth learning now

Operations stack. Everything in the Recipe panel runs top to bottom, so you can chain Extract IP addresses into Sort into Defang IP Addresses and get a finished IOC list in one pass.

The Output updates as you type. There is no run button to press and nothing to submit.

To start over, click the bin icon at the top of the Recipe panel. Leftover operations from a previous exercise are the usual reason output looks wrong.

A.4 If something is not working

Problem	What to do
My SHA-256 output does not match the printed value	You have an extra space or line break in the Input. Clear it with Ctrl+A then Delete and type the line by hand. The character count at the bottom of the Input panel should read 34.
CyberChef output looks wrong or empty	An operation from a previous exercise is still in the Recipe panel. Click the bin icon at the top of the Recipe panel to clear it, then add only the operation you need.
CyberChef will not load at all	Use the built-in operating system commands in Section 8.5 for hashing. For decoding base64 and extracting addresses, the printed evidence is short enough to read by hand. Note the substitution in your report.
The NVD page is slow or will not load	Use the printed ranges in Appendix A.1 and say so in your report. Retry NVD later if you want to confirm.
I cannot find a CVE in the KEV search box	Check for a typo first, then confirm against Appendix A.2. Not finding a CVE is often the correct answer - one of the nine is genuinely not in the catalog.

Problem	What to do
I do not know which findings to validate	Start with the three Critical 9.8 findings that have CVE numbers. Section 4.5 points you at each of them without giving the answer away.
I am stuck on Day 3 and cannot see the chain	Work backwards. Start at the alert in Section 7.1, find the process in Section 7.3, find where that process came from in Section 9.3, then find who created it in Section 10.2.

A.5 Working with an AI assistant

You may use an AI assistant on this lab, and you should. Use it the way a working analyst does.

- **Good use** - ask it to explain what a pkexec log line means, what a base64 string is, why a UID 0 warning matters, or to review whether your executive summary is clear to a non-technical reader.
- **Bad use** - asking it which findings are false positives. You will get an answer that sounds confident and you will not be able to defend it, because you did not do the version comparison yourself.
- **Always verify** - AI assistants are frequently wrong about specific CVE version ranges and KEV membership. Every factual claim about a CVE must be confirmed against NVD or the CISA catalog before it goes in your report. That verification habit is itself part of what Week 7 covers.

Cite it. If an AI helped you reach a conclusion, note where and what you did to verify it. That is the standard a regulated bank holds analysts to, and it is the standard here.